# Rechnerstrukturen Teil II

## *Einleitung und ISA*

Dr.-Ing. Christian Hakert / Dr. rer. nat. Mario Günzel

TU Dortmund, Fakultät für Informatik, Lehrstuhl 12



by Decker

# Kontext

Die Wissenschaft Informatik befasst sich mit der Darstellung, Speicherung, Übertragung und Verarbeitung von Informationen

[Gesellschaft für Informatik]

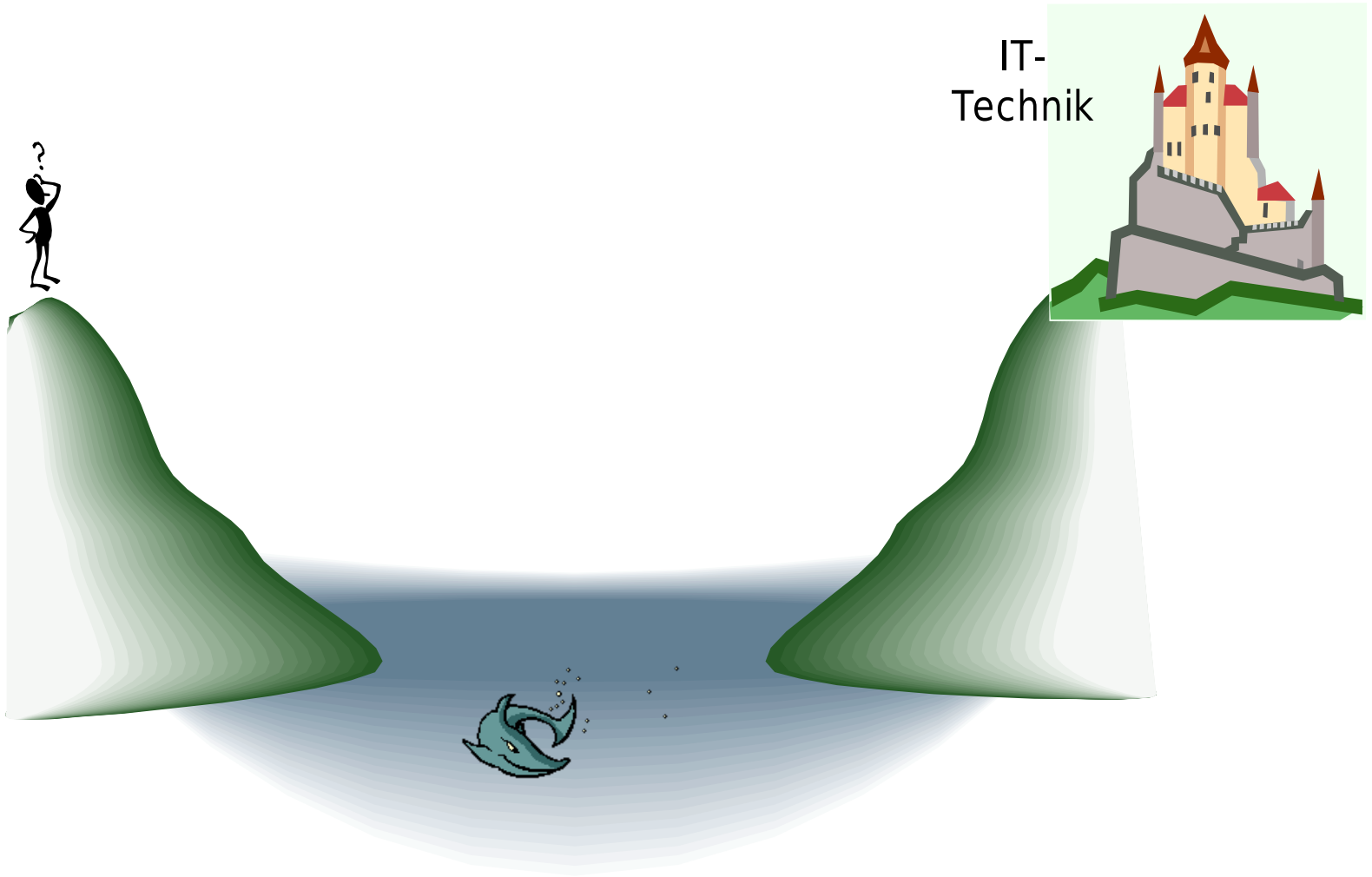
→ Darstellung von Informationen: z.B. Zahlendarstellung in RS, Teil 1

→ Verarbeitung von Informationen: Inhalt von RS, Teil 2

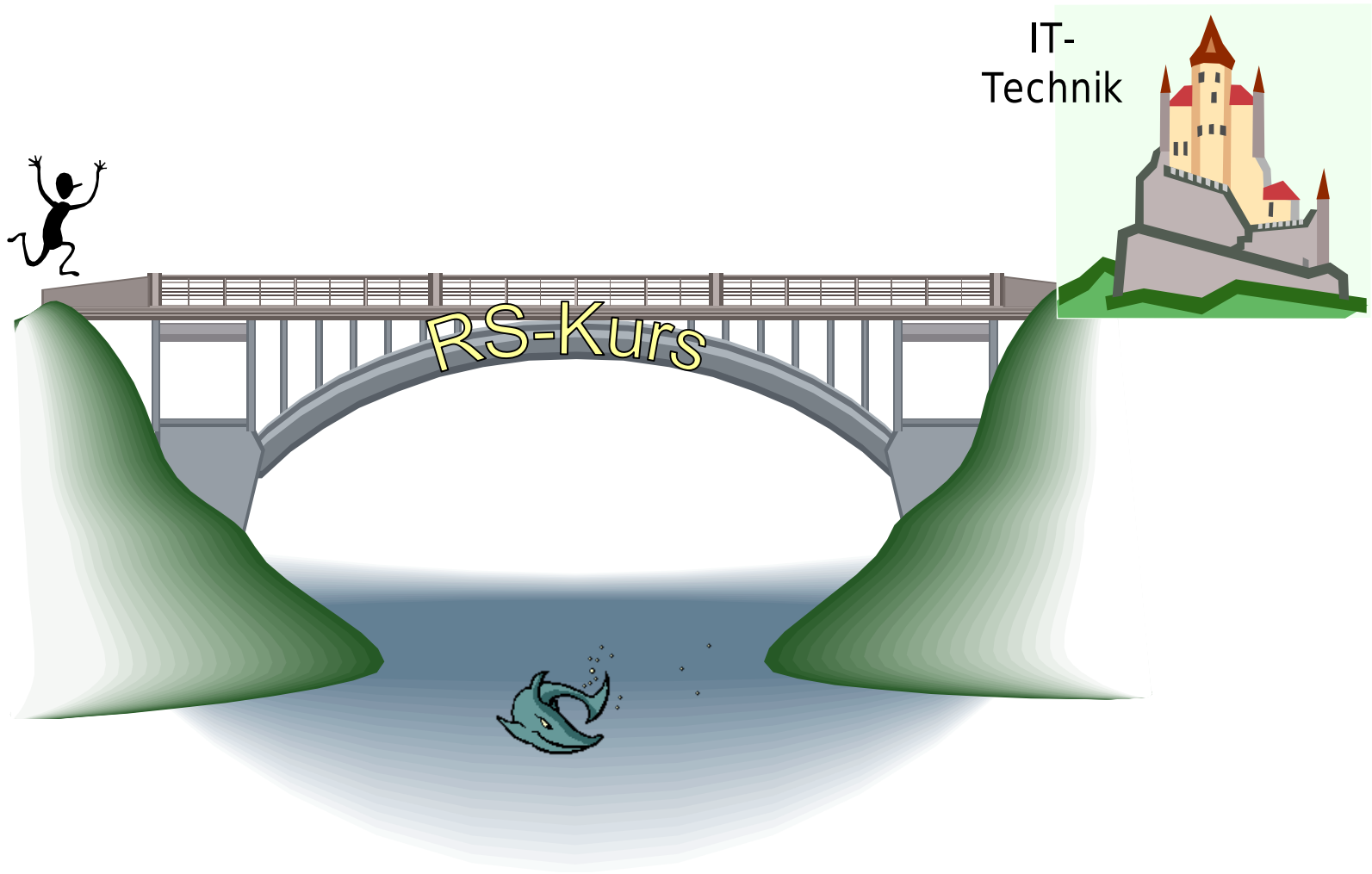
# Motivation

- Jede Ausführung von Programmen bedarf einer zu Ausführung fähigen Hardware
- Wir nennen diese auch Ausführungsplattformen (**execution platforms**)
- Platform-based design ist ein Ansatz für viele Anwendungen (Handys, Autos,...)
- Plattformen sind nicht immer ideal (z.B führen Anwendungen nicht in 0 Zeit mit 0 Energie aus)
- Grundlegendes Verständnis für nicht-ideales Verhalten ist wichtig
- Deshalb Beschäftigung in dieser Vorlesung mit Ausführungsplattformen

# Problematische Situationen …



# … und Techniken zu deren Vermeidung



# Thema des zweiten Teils der Vorlesung

**Ziel: Verständnis der Arbeitsweise von Rechnern einschließlich**

- der Programmierung von Rechnern
- des Prozessors
- der Speicherorganisation
- des Anschlusses von Peripherie
- Anwendungen bei Eingebetteten Systemen

# Stil des zweiten Teils der Vorlesung

**Stil:**

- Betonung des Skripts
- Integration mit praktischen Übungen
  Einsatz eines Simulators
- Buch von Hennessy/Patterson: Computer Organization and Design: The Hardware/Software Interface RISC-V Edition, 2018
- Weitere Bücher: siehe Literaturverzeichnis im Skript

# Rechnerarchitektur -Einleitung-

## Abstraktionsebenen

| **Ebene** | **Beispiel** | **Ausführung** | **Modul** |
|---|---|---|---|
| Übliche Programmiersprache | `for`, `while`, `if` | Compiler | DAP |
| Assemblerprogramm-Ebene | `bne x1,x2,loop` | Compiler/Assembler | RS2 |
| Maschinenprogramm-Ebene/Befehlsschnittstelle | $01010...10_2$ | Assembler | RS2 |
| Reg-Transfer-Verhaltensebene | `Reg[2]:=Reg[3]` | | RS2 |
| Reg-Transfer-Strukturebene | Addierer, Multiplexer, Register | | RS2 |
| Gatter-Ebene | $f=a \vee bc$ | | RS1 |
| Transistorebene | | | ETKT |

# 2.2 Die Befehlsschnittstelle
# 2.2.1 Der RISC-V-Befehlssatz

**Beispiel: RISC-V** (~ **R**educed **I**nstruction **S**et **C**omputer **V**)
Vorgestellt im Jahr 2010, (mit **32** Bits)

**Warum RISC-V?**

- Weitgehend sauberer und klarer Befehlssatz
- Kein historischer Ballast
- Basis der richtungsweisenden Bücher von Hennessy/Patterson
- Simulator verfügbar
- RISC-V Prozessoren werden realisiert (z.B. SiFive)

# Begriffe

- **RISC-V-Befehle** sind elementare Anweisungen an die RISC-V-Maschine (z.B. SiFive)
- Ein **RISC-V-Maschinenprogramm** ist eine konkrete Folge von RISC-V-Befehlen
- Die **RISC-V-Maschinensprache** ist die Menge möglicher RISC-V-Maschinenprogramme
- Entsprechendes gilt für die Assemblerprogramm-Ebene
- Vielfach keine Unterscheidung zwischen beiden Ebenen

# RISC-V-Assembler-Befehle
# Arithmetische und Transportbefehle

## Erstes Beispiel: Addition

**Allgemeines Format:**

**`add a,b,c`** mit **`a,b,c ∈ x0,...,x31`**

**`x0,...,x31`** stehen für 32 Bit lange Register

**`a`** ist das Zielregister

**Beispiel:**

```
add x3, x2, x1
```

# Funktion des Additionsbefehls

**Beispiel:add x3, x2, x1**

| Register (“Reg”) | Inhalt |
|---|---|
| x0 | “0…0” |
| x1 | 5 |
| x2 | 6 |
| x3 | 11 |
| … | |
| x31 | |

**x0** ist ständig 0

# Simulation des RISC-V-Befehlssatzes mit RARS

Simulator **RARS** (erfordert Java)

Warum Simulation statt Ausführung des Intel-Befehlssatzes?

- Möglichkeit der Benutzung des RISC-V-Befehlssatzes
- Keine Gefährdung des laufenden Systems
- Bessere Interaktionsmöglichkeiten
- RARS läuft auf verschiedenen Plattformen: Windows, MAC OS X, Linux

Installation jetzt dringend empfohlen, für Studierende ohne PC-Zugang Nutzung an Fakultätsrechnern möglich.

Schreiben Sie eigene kleine Programme!

# Semantik: per Register-Transfer-Notation (genauer: Register-Transfer-Verhaltens-Notation)

Argumente oder Ziele: Register oder Speicher, z.B. **`Reg[3]; PC; Reg[31]`**

Zuweisungen: mittels :=, z.B.
**`PC:=Reg[31]`**

Konstante Bitvektoren: Einschluss in “, z.B.:
**“01010100011”**

Selektion von einzelnen Bits: runde Klammern:
**`PC(15:0)`** –VHDL **`downto`** statt `:´, alt: `.´ vor `(´

Konkatenation (Aneinanderreihung) mit &, z.B.

```
(x1 & x2);
PC := PC(31:28) & I(25:0) & "00"
```

# Semantik des Additionsbefehls

**Beispiel:**

```
add x3,x2,x1 # Reg[3]:= Reg[2]+Reg[1]
```

# leitet in der Assemblernotation einen Kommentar ein

Register speichern (Teils des) aktuellen Zustands; **add**-Befehl veranlasst Zustandstransformation.

# Addition für 2k-Zahlen oder für vorzeichenlose Zahlen? (1)

Muss der **add**-Befehl “linkestes” Bit als Vorzeichen betrachten?

Nein, bei 2k-Zahlen und bei vorzeichenlosen Zahlen (“Betragszahlen”) kann jede Stelle des Ergebnisses gemäß der Gleichungen für Volladdierer bestimmt werden, unabhängig davon, ob die Bitfolgen als 2k-Zahlen oder als Betragszahlen zu interpretieren sind (siehe RS, Teil 1).

Es reicht ein Additionsbefehl zur Erzeugung der Ergebnis-Bitvektoren für beide Datentypen aus. Allerdings Unterschiede hinsichtlich der Behandlung von Bereichsüberschreitungen.

# Das Speichermodell der RISC-V-Architektur

Registerspeicher (**Reg**)

← 4 Bytes →

| | | | | |
|---|---|---|---|---|
| x0 | | | | |
| x1 | | | | |
| x2 | | | | |
| x.. | | | | |
| x30 | | | | |
| x31 | | | | |

Hauptspeicher (**Speicher**)

← 4 Bytes →

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 3 |
| 4 | | | | |
| 8 | | | | |
| | | | | |
| | | | | |
| ... | | | | |
| $\leq 2^{32}-4$ | | | | |

+ spez. Register (PC)

# Das Speichermodell der RISC-V-Architektur

Registerspeicher (**Reg**)

← 4 Bytes →

| | | | | |
|---|---|---|---|---|
| x0 | | | | |
| x1 | | | | |
| x2 | | | | |
| x.. | | | | |
| x30 | | | | |
| x31 | | | | |

Hauptspeicher (**Speicher**)

← 4 Bytes →

| | | | | |
|---|---|---|---|---|
| 0 | 3 | 2 | 1 | 0 |
| 4 | | | | |
| 8 | | | | |
| | | | | |
| | | | | |
| ... | | | | |
| $\leq 2^{32}-4$ | | | | |

+ spez. Register (PC)

# Eigenschaften des Von-Neumann-Rechners (1)

Wesentliche Merkmale der heute üblichen Rechner, die auf dem Prinzip des Von-Neumann-Rechners basieren:

1. Einteilung des Speichers in Zellen gleicher Größe, die über **Adressen** angesprochen werden können.

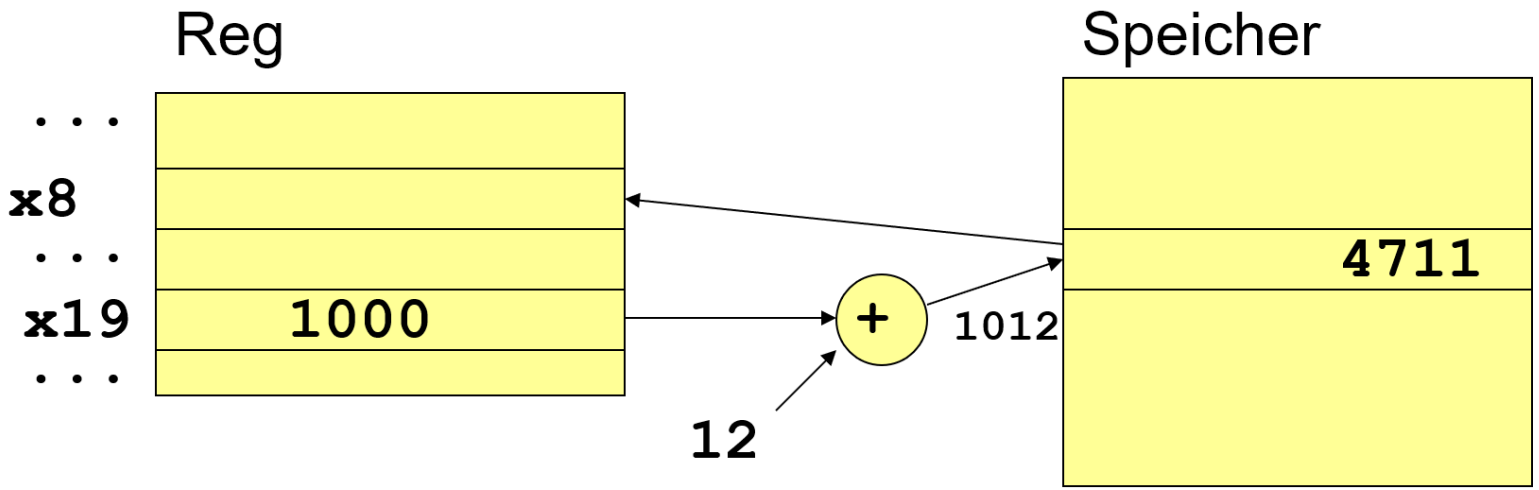
# Der `load word`-Befehl

Allgemeine Form:

**`lw ziel`**, **`offset(reg)`** mit **`ziel`**, **`reg`** $\in$ **`x0...x31`**, **`offset`**: Konstante $\in -2^{11} \ldots 2^{11} - 1$, deren Wert beim Laden des Programms bekannt sein muss, bzw. deren Berzeichner.

Beispiel:

```
lw x8, 12(x19) # Reg[8]:= Speicher[12+Reg[19]]
```


Reg
Speicher
. . .
x8
. . .
x19
. . .
1000
4711
+
1012
12

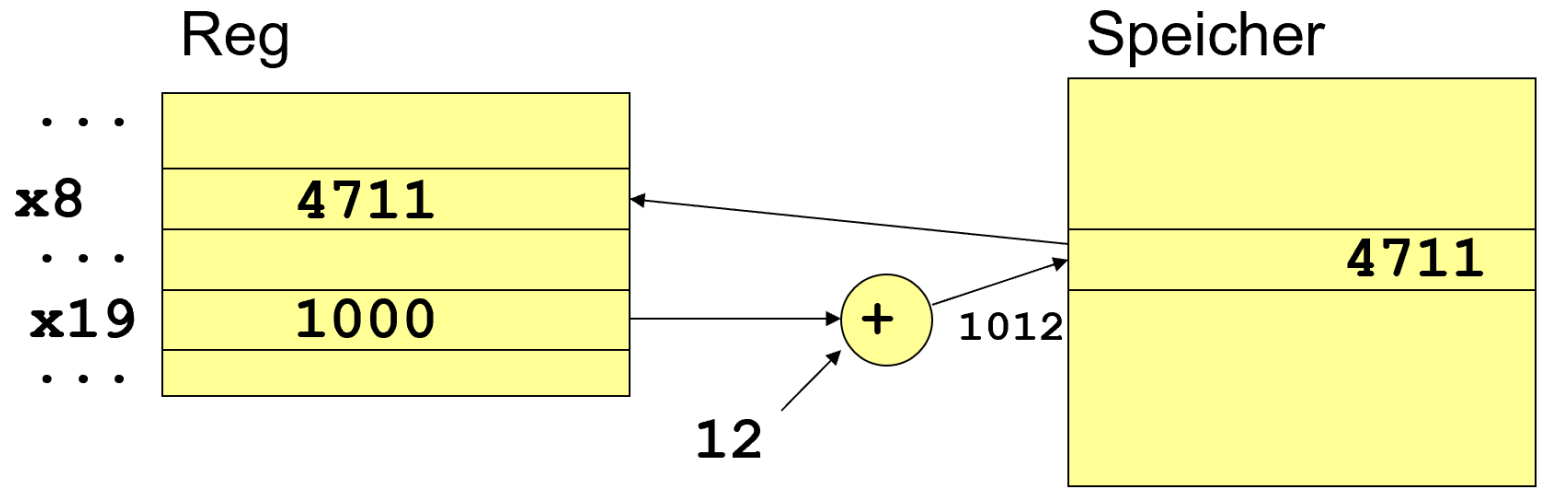
# Der `load word`-Befehl

Allgemeine Form:

**`lw ziel`**, **`offset(reg)`** mit **`ziel`**, **`reg`** $\in$ **`x0...x31`**, **`offset`**: Konstante $\in -2^{11} \ldots 2^{11} - 1$, deren Wert beim Laden des Programms bekannt sein muss, bzw. deren Berzeichner.

Beispiel:

```
lw x8, 12(x19) # Reg[8]:= Speicher[12+Reg[19]]
```


Reg
Speicher
. . .
x8
4711
. . .
x19
1000
. . .
+
1012
12
4711

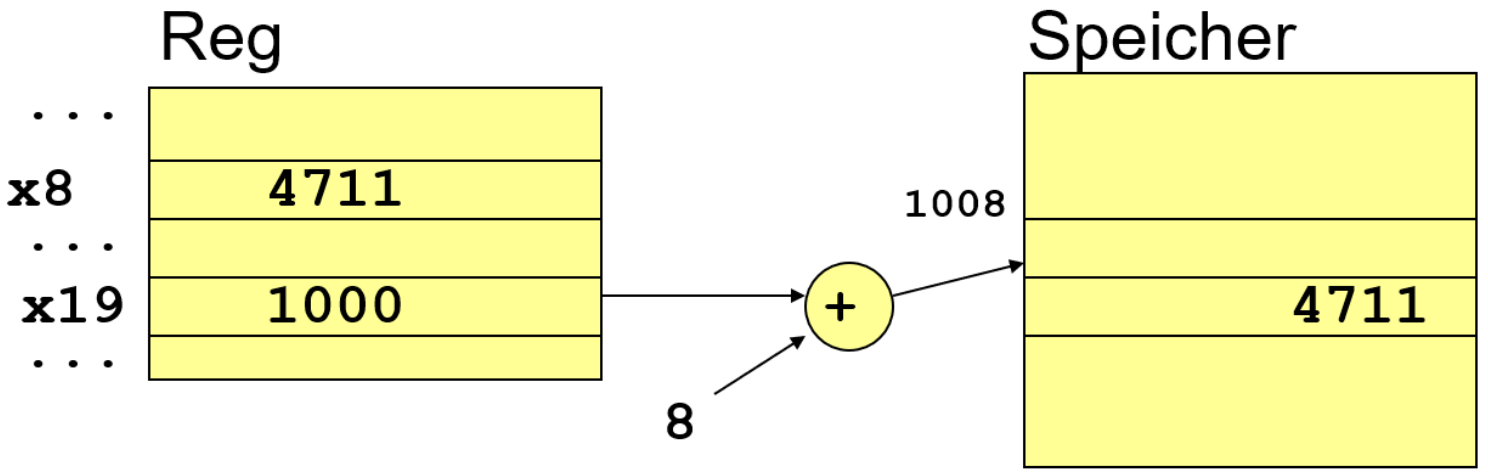
# Der `store word`-Befehl

Allgemeine Form:

**`sw quel, offset(reg)` mit `quel,reg` ∈ `x0...x31`, `offset`**: Konstante ∈ $-2^{11} \ldots 2^{11} - 1$, deren Wert beim Laden des Programms bekannt sein muss, bzw. deren Berzeichner.

Beispiel:

$\Longrightarrow$ **`sw x8, 8(x19) # Speicher[8+Reg[19]] := Reg[8]`**



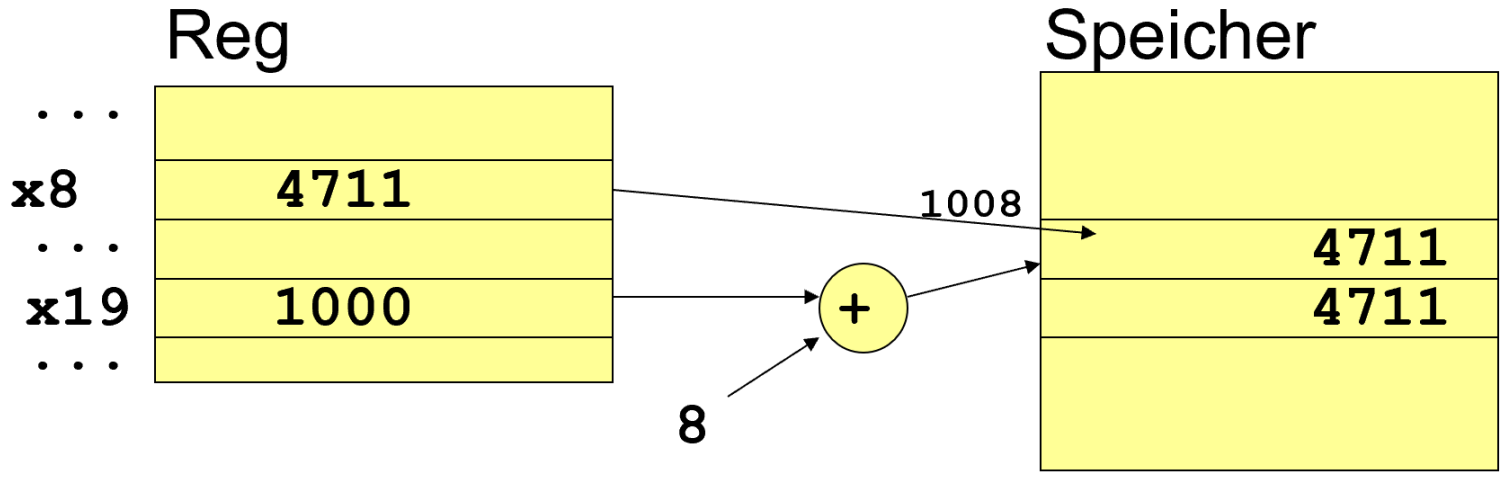
# Der `store word`-Befehl

Allgemeine Form:

**`sw quel, offset(reg)` mit `quel,reg` ∈ `x0...x31`, `offset`**: Konstante $\in -2^{11} \ldots 2^{11} - 1$, deren Wert beim Laden des Programms bekannt sein muss, bzw. deren Berzeichner.

Beispiel:

$\Longrightarrow$ **`sw x8, 8(x19) # Speicher[8+Reg[19]] := Reg[8]`**



# Darstellung von Befehlen im Rechner

Zergliederung eines Befehlswortes in **Befehlsfelder**; Jede benutzte Zergliederung heißt **Befehlsformat**;

Beispiele:

| | **Größe[Bit]** | **7** | ***5*** | ***5*** | **3** | ***5*** | **7** |
|---|---|---|---|---|---|---|---|
| Arithmetik | Format R | funct7 | rs2 | rs1 | funct3 | rd | op |
| Immediate (lw) | Format I | imm(11:0) | | rs1 | funct3 | rd | op |
| Store | Format S | imm(11:5) | rs2 | rs1 | funct3 | imm(4:0) | op |

function(funct): kodiert Befehlsvariante

Operationscode(op): kodiert Befehl

# Speicherung von Befehlen im Rechner

**Assemblerprogramm**

```
lw x2, 100(x0)
lw x3, 104(x0)
add x3, x2, x3
sw x3, 108(x0)
```

Übersetzung des Assemblerprogramms in ein ladbares Maschinenprogramm ist Aufgabe des Assemblers

`funct3`

`funct7`

Hauptspeicher (**Speicher**)

| | |
|---|---|
| 0 | 100 0 2 2 3 |
| 4 | 104 0 2 3 3 |
| 8 | 0 3 2 0 3 51 |
| 12 | 96 3 0 2 12 35 |
| ... | |
| 100 | 15 |
| 104 | 10 |
| 108 | |

| | |
|---|---|
| . . . | $\leq 2^{32}-4$ |

# Eigenschaften des Von-Neumann-Rechners (2)

Wesentliches Merkmale der heute üblichen Rechner, die auf dem Prinzip des Von-Neumann-Rechners basieren:

2. Verwendung von speicherprogrammierbaren Programmen
3. Speicherung von Programmen und Daten in demselben Speicher

# Zusammenfassung

- Schichtmodell
  - Programme in höherer Programmiersprache
  - Assemblerprogramme
  - Maschinenprogramme
  - RT-Verhalten/-Strukturen
  - Gatter
- Unterscheidung zw. Befehlen, Programmen, Sprachen
- Exemplarische Betrachtung der RISC-V-Assembler- & Maschinensprache
  - `add`-, `lw`-, `sw`-Befehle; RT-Semantik
  - Speichermodell
  - Darstellung von Befehlen Prinzipien der von Neumann-Maschine

# Abarbeitung: immer der Reihe nach

**Reg**

| | |
|---|---|
| `x0` | |
| `x1` | |
| `x2` | |
| `x2` | 15 |
| `x3` | 10 |
| `...` | 0 |

Hauptspeicher (**`Speicher`**)

Der Zeiger auf den gerade ausgeführten Befehl wird im Programmzähler PC (*program counter*) gespeichert

| | |
|---|---|
| 0 | 100 0 2 2 3 |
| 4 | 104 0 2 3 3 |
| 8 | 0 3 2 0 3 51 |
| 12 | 96 3 0 2 12 35 |
| | |
| 100 | 15 |
| 104 | 10 |
| 108 | |
| | |
| $\leq 2^{32}-4$ | |

# Abarbeitung: immer der Reihe nach

**Reg**

| | |
|---|---|
| `x0` | |
| `x1` | |
| `x2` | |
| `x2` | 15 |
| `x3` | 25 |
| `...` | 0 |

Hauptspeicher (**`Speicher`**)

Der Zeiger auf den gerade ausgeführten Befehl wird im Programmzähler PC (*program counter*) gespeichert

| | |
|---|---|
| 0 | 100 0 2 2 3 |
| 4 | 104 0 2 3 3 |
| 8 | 0 3 2 0 3 51 |
| 12 | 96 3 0 2 12 35 |
| | |
| 100 | 15 |
| 104 | 10 |
| 108 | |
| | |
| $\leq 2^{32}-4$ | |

# Eigenschaften des Von-Neumann-Rechners (3)

Wesentliches Merkmale der heute üblichen Rechner, die auf dem Prinzip des Von-Neumann-Rechners basieren:

4. Die sequentielle Abarbeitung von Befehlen
5. Es gehört zur Semantik eines jeden Befehls (Ausnahme: Sprungbefehle, s.u.), dass zusätzlich zu den erwähnten Änderungen der Speicherinhalte noch der Programmzähler PC erhöht wird, im Fall der RISC-V-Maschine jeweils um 4

# 2.2.1 Der RISC-V-Befehlssatz
# 2.2.1.1 Arithmetische und Transportbefehle

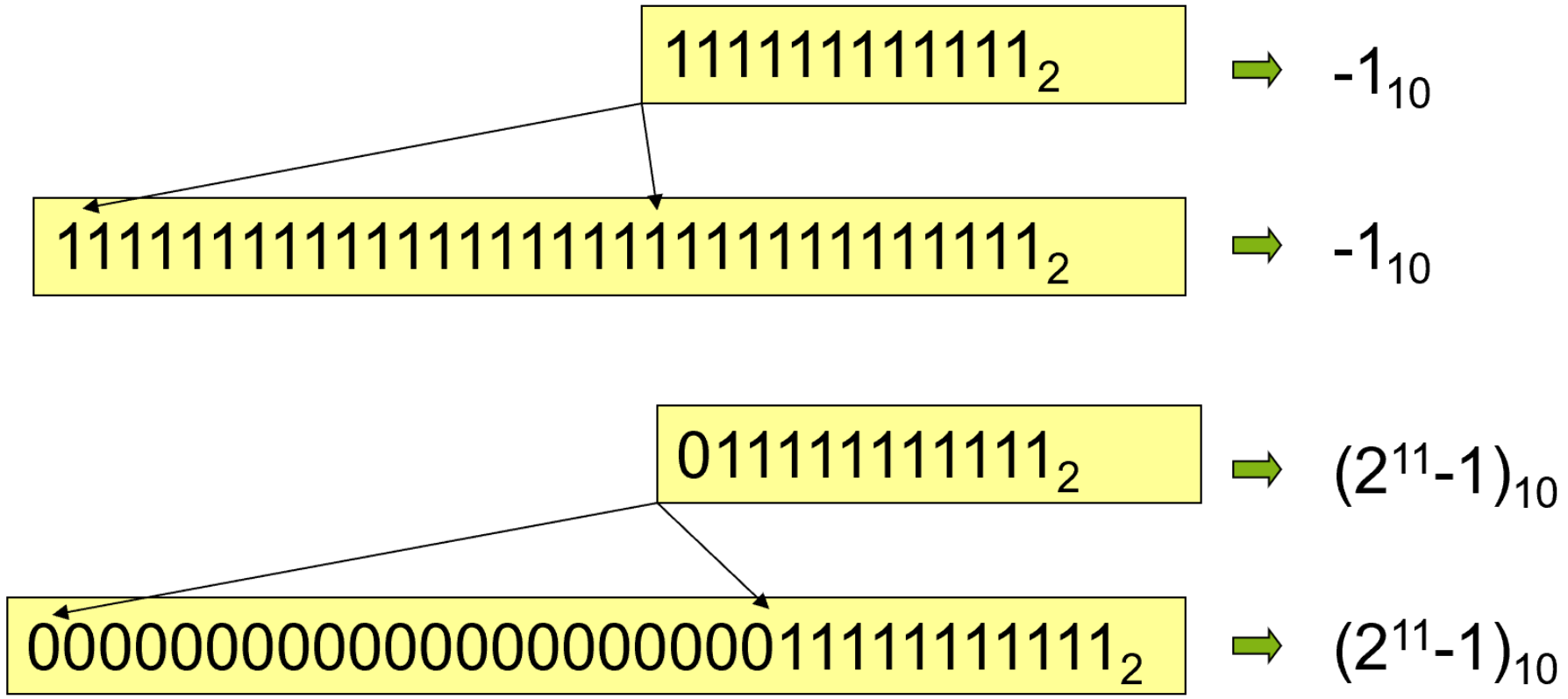
Darstellung von Befehlen im Rechner (Wdh.)

| | **Größe[Bit]** | **7** | **5** | **5** | **3** | **5** | **7** |
|---|---|---|---|---|---|---|---|
| Arithmetik | Format R | funct7 | rs2 | rs1 | funct3 | rd | op |
| Immediate (lw) | Format I | imm(11:0) | | rs1 | funct3 | rd | op |
| Store | Format S | imm(11:5) | rs2 | rs1 | funct3 | imm(4:0) | op |

Problem: Passt nicht zur Länge der Register

# Nutzung des 12-Bit-Offests in der 32-Bit Arithmetik

**Replizierung** des Vorzeichensbits liefert 32-Bit 2k-Zahlen



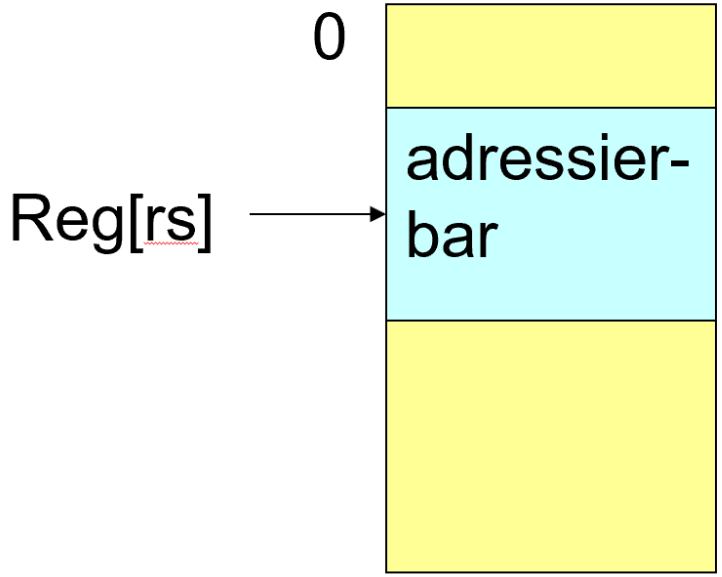

`sign_ext(a,m)`: Funktion, die Bitvektor a auf `m` Bits erweitert

# Auswirkung der Nutzung von `sign_ext` bei der Adressierung

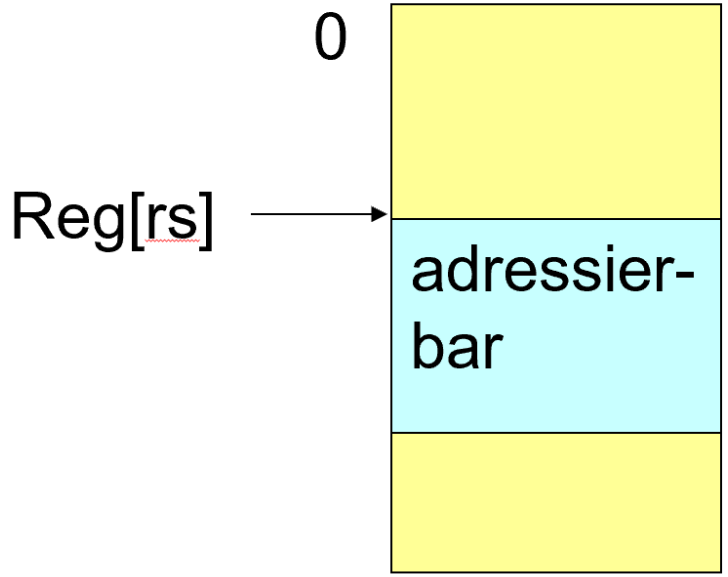
Präzisere Beschreibung der Bedeutung des `sw`-Befehls:

```
sw rt, offset(rs) # Speicher[Reg[rs]]+sign_ext(offset, 32)] := Reg[rt]
```

mit Vorzeichenweiterung




mit *zero-extend* (“00..00” & ...)



# Einsatz der Vorzeichenerweiterung bei Direktoperanden (immediate addressing)

- Beschreibung der Bedeutung des `add immediate`-Befehls

  **`addi rt, rs, const #`** benutzt Format I

  **`# Reg[rt] := Reg[rs] + sign_ext(const,32)`**

Format: I

# Subtraktionsbefehle

**`sub x4,x3,x2 # Reg[4] := Reg[3]-Reg[2]`**

Format: R

**`subi`** Befehl gibt es nicht

Stattdessen: Addition mit negativem Immediate

**`addi x4, x3, -45`**

# Multiplikationsbefehle (1)

Die Multiplikation liefert doppelt lange Ergebnisse.

Beispiel: $-2^{31} * -2^{31} = 2^{62}$;

$2^{62}$ benötigt zur Darstellung einen 64-Bit-Vektor.

Wo soll man ein solches Ergebnis abspeichern?

RISC-V-Lösung: separate Befehle für oberen/unteren 32 Bit

```
mul x4,x3,x2 # Reg[4] := Reg[3]*Reg[2]
```

in Register 4 wird der Niederwertige Teil des Ergebnisses abgespeichert.

**Keine Unterscheidung nach signed/unsigned nötig.**

# Multiplikationsbefehle (2)

Unterschiedliche Multiplikationsbefehle für die oberen 32 Bit nötig:

- signed * signed **`mulh x4,x3,x2`**
- unsigned * unsigned **`mulhu x4,x3,x2`**
- signed * unsigned **`mulhsu x4,x3,x2`**

Woher wissen wir, wann ein Überlauf stattgefunden hat?
Unterscheidung, ob das Ergebnis signed (**`mulh`**/**`mulhsu`**) oder unsigned (**`mulhu`**) ist

# Multiplikationsbefehle (3)

Ist das Ergebnis der Multiplikation **unsigned**, hat ein Überlauf stattgefunden, wenn mindestens ein Bit der oberen 32 Bit ungleich 0 ist.

| | | | |
|---|---|---|---|
| `0xffffffff *` | `0x00000010` | `= 0x00000001` | `fffffffe` |
| | | Obere 32 Bit ≠0 - Überlauf hat stattgefunden | |

Ist das Ergebnis der Multiplikation **signed**, hat ein Überlauf stattgefunden, wenn die oberen 32 Bit nicht nur Vorzeichenbits sind.

| | | | |
|---|---|---|---|
| `0xfffffffe *` | `0x7fffffff` | `= 0xfffffffe` | `e0000002` |
| Vorzeichenbit 1 - Zahl negativ | Vorzeichenbit 0 - Zahl positiv | Obere 32 Bit ≠1 - Überlauf hat stattgefunden | Vorzeichenbit1 |

# Divisionsbefehle (1)

Division zweier Zahlen (immer abgerundet):

- **`div x4,x3,x2 #`** für ganze Zahlen in 2k-Darstellung
  **`# Reg[4] :=[3]/Reg[2]`**
- **`divu x4,x3,x2 #`** für natürliche Zahlen (*unsigned integers*)
  **`# Reg[4] :=Reg[3]/_u Reg[2]`**

Das Ergebnis wird dabei immer abgerundet (ganzzahlige Division)

Das Vorzeichen wird dabei analog zur Multiplikation ermittelt

# Divisionsbefehle (2)

Zur Ermittlung des Rests hat RISC-V eigene Befehle

- **`rem x4,x3,x2 #`** für ganze Zahlen in 2k Darstellung
  **`# Reg[4] :=Reg[3]-Reg[2]*(Reg[3]/Reg[2])`**
- **`remu x4,x3,x2 #`** für natürliche Zahlen (unsigned integers)
  **`# Reg[4] :=Reg[3]-Reg[2]*(Reg[3]/_u Reg[2])`**

Das Vorzeichen wird dabei vom Dividenden übernommen

# Divisionsbefehle (3)

Überlauf möglich mit:

- **`0x80000000 / 0xffffffff # signed-Division der größten negativen Zahl durch -1`**

Eine Division durch 0 ist ebenfalls möglich

Es gilt:

| | Dividend | Divisor | `divu` | `remu` | `div` | `rem` |
|---|---|---|---|---|---|---|
| Div. durch 0 | x | $0$ | $2^{32}-1$ | x | -1 | x |
| Überlauf | $-2^{31}$ | $-1$ | - | - | $-2^{31}$ | $0$ |

# Logische Befehle

| Beispiel | Bedeutung | Kommentar |
|---|---|---|
| `and x4,x3,x2` | `Reg[4] :=Reg[3]` $\wedge$ `Reg[2]` | und |
| `or x4,x3,x2` | `Reg[4] :=Reg[3]` $\vee$ `Reg[2]` | oder |
| `andi x4,x3,100` | `Reg[4] :=Reg[3]` $\wedge$ `100` | und mit Konstanten |
| `slli x4,x3,10` | `Reg[4] :=Reg[3] << 10` | Schiebe logisch nach links |
| `srli x4,x3,10` | `Reg[4] :=Reg[3] >> 10` | Schiebe logisch nach rechts |

zero_ext

# Laden von Konstanten (1)

Wie kann man 32-Bit-Konstanten in Register laden?

- Direktoperanden für die unteren 12 Bit:
  **`ori r,s,const`** `# Reg[r]:=Reg[s] ∨ (00000`$_{16}$ `& const)`
- Für den Sonderfall s=x0:
  **`ori r,x0,const`** `# Reg[r]:=0 ∨ zero_ext(const,32)`

| 0 | const |
|---|---|
| Vorzeichen(const) | const |

# Laden von Konstanten (2)

- Für Konstanten mit unteren 12 Bit = 0

  **`lui r, const`** `# Reg[r]:=const &000`$_{16}$

  (load upper immediate)

| 0 | const |
|---|---|

| | **Größe [Bit]** | **20** | **5** | **7** |
|---|---|---|---|---|
| Upper-Immediate (lui) | Format U | imm(31:12) | rd | op |

# Laden von Konstanten (3)

- Für andere Konstanten:

```
lui x1, const' # 20 Bits
ori r,x1,const'' # 12 Bits
```

| | 20 Bits | 12 Bits |
|---|---|---|
| | const’ | 0 |
| v | 0 | const’’ |
| = | const’ | const’’ |

Sequenz wird vom Assembler für `li` (load immediate) erzeugt.

# Der load adress - Befehl `la`

In vielen Fällen muss die Adresse einer Speicherzelle in einem Register bereit gestellt werden.

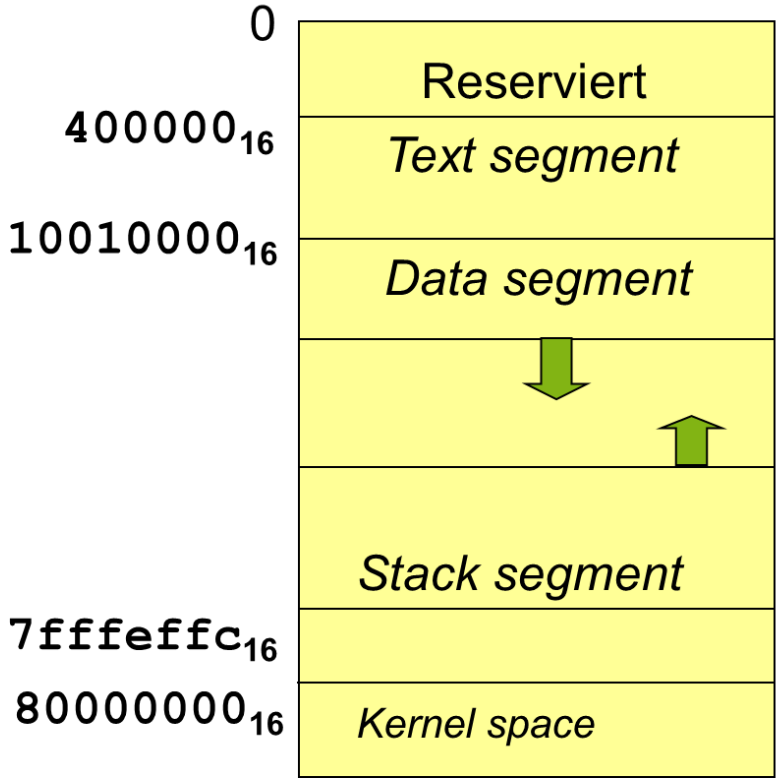
Dies ist mit den bislang vorgestellten Befehlen zwar möglich, der Lesbarkeit wegen wird ein eigener Befehl eingeführt.

Der Befehl **`la`** entspricht dem **`lw`**-Befehl, wobei der Speicherzugriff unterbleibt.

Beispiel:

```
la x2, label # Reg[2] := adress(label)
```

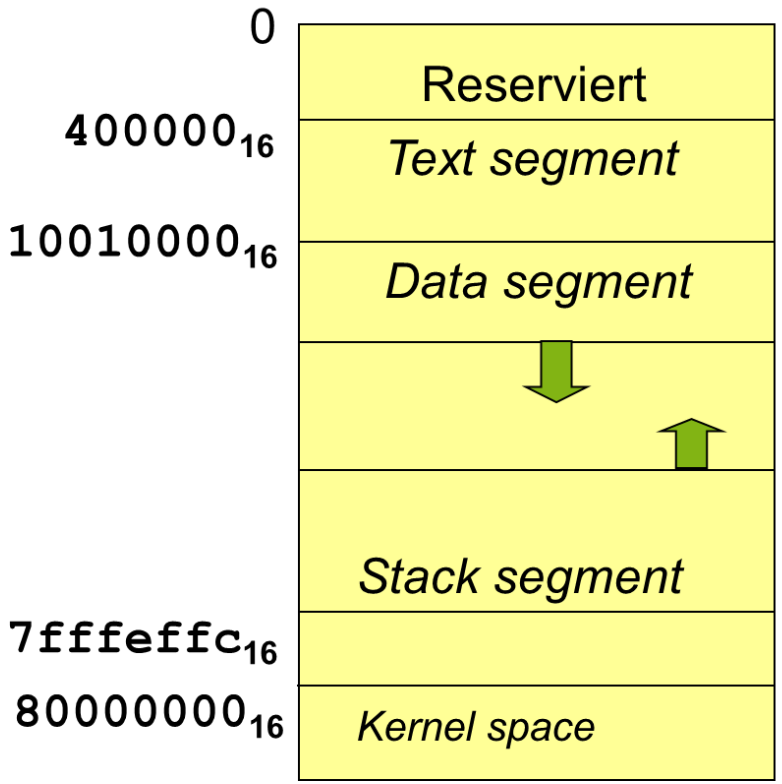
# Einteilung des Speicherbereichs (memory map) im Simulator folgt üblicher C/Unix-Konvention




Befehle
Daten bekannter Größe + weitere Daten

Dynamisch erzeugte Daten, Auf- und Abbau bei Aufruf von Prozeduren/Unterprogrammen
Adresse Ein-/ Ausgabegeräte
Betriebssystem

# Problem der Nutzung nicht zusammenhängender Adressbereiche




Mann müsste den Segmenten verschiedenen physikalischen Speicher zuordnen, wenn man mit diesen Adressen wirklich den physikalischen Speicher adressieren würde.

Auswege:

- Umrechnung der Adressen (siehe Abschnitt über Speicher) oder
- Benutzung weitgehen zusammenhängender Speicherbereiche

# Benutzung weitgehend zusammenhängender Speicherbereiche

**Betriebssystem**

---

**Text segment**

---

**Data segment**

---

**Stack segment**

---

...

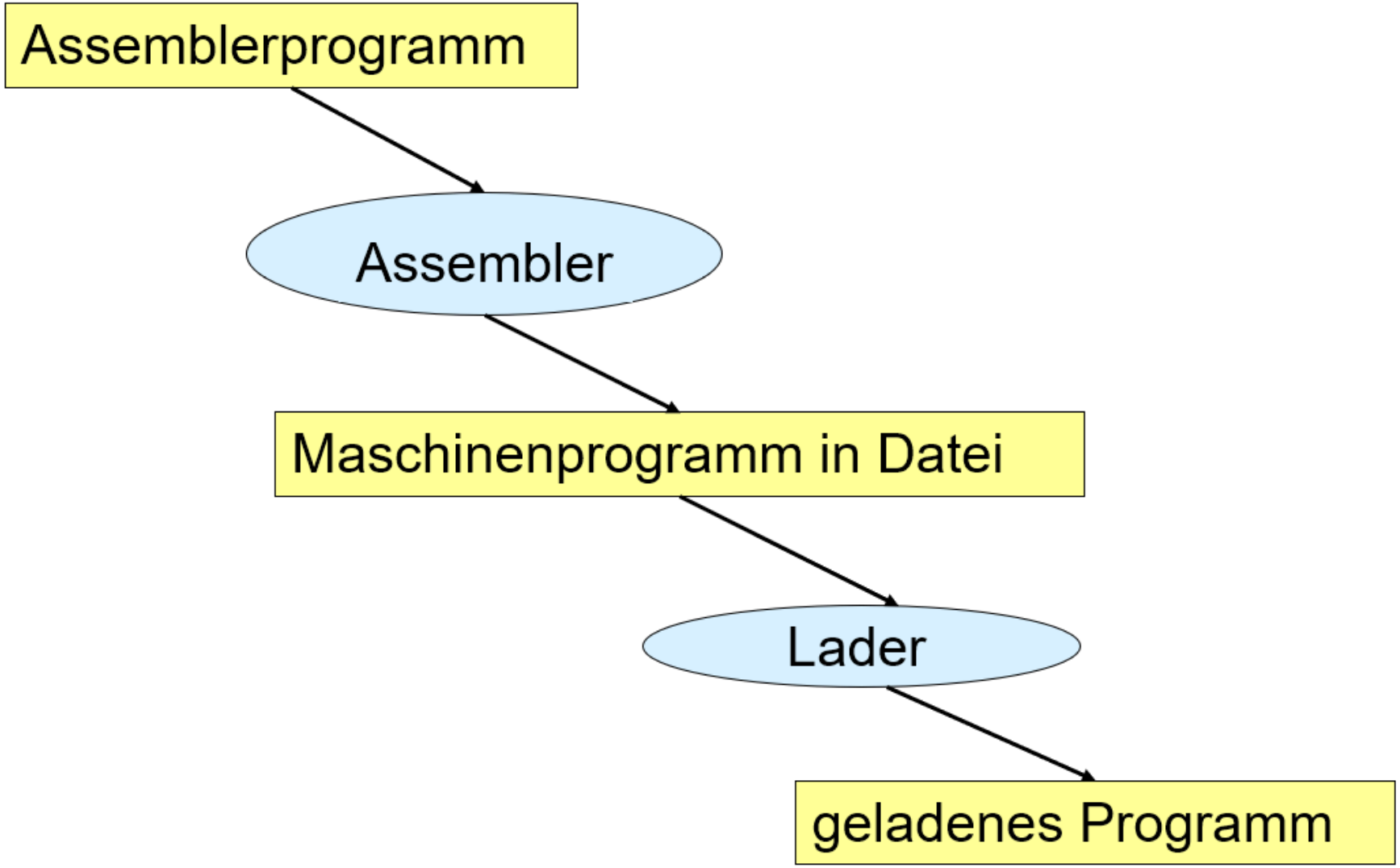
Erfordert eine relativ gute Kenntnis der Größe der Segmente
Kann beim RARS konfiguriert werden (→ Einstellungen)

# Benutzung der Speicherbereiche in einem Beispielprogramm

## Beispielprogramm

```
      .globl main                #Globales Symbol
main: lui x1, 0x10010             #Anfang Datenbereich
      lw x2, 0x00000000(x1)
      lw x3, 0x00000004(x1)
      add x3,x2,x3
      sw x3, 0x00000008(x1)
      ...ecall
```

# Transformation der Programmdarstellungen



# Funktion des Assemblers (1)

- Übersetzt symbolische Befehlsbezeichnungen in Bitvektoren
- Übersetzt symbolische Registerbezeichnungen in Bitvektoren
- Übersetzut Pseudo-Befehle in echte Maschinenbefehle
- Verwaltet symbolische Marken
- Nimmt die Unterteilung in verschiedene Speicherbereiche vor

# Funktion des Assemblers (2)

Verarbeitet einige Anweisungen an den Assembler selbst:

| | |
|---|---|
| `.ascii` **text** | Text wird im Datensegment abgelegt |
| `.asciiz` **text** | Text wird im Datensegment abgelegt, 0 am Ende |
| `.data` | die nächsten Worte sollen in das Daten-Segment |
| `.extern` | Bezug auf externes globales Symbol |
| `.globl` *id* | Bezeichner *id* soll global sichtbar sein |
| `.space` *n* | *n* Bytes im Daten-Segment reservieren |
| `.text` | die nächsten Worte kommen in das Text-Segment |
| `.word` *wert*,... | *wert* Im aktuellen Bereich zu speichern |

# Assembler übersetzt symbolische Registernummern”

| Register | Name | Bedeutung | Register | Name | Bedeutung |
|---|---|---|---|---|---|
| x0 | zero | Always 0 | x8 | s0/fp | Saved register/ frame pointer |
| x1 | ra | Return address | x9 | s1 | Saved register |
| x2 | sp | Stack pointer | x10-x11 | a0-a1 | Function arguments/ return values |
| x3 | gp | Global pointer | x12-x17 | a2-a7 | Function arguments |
| x4 | tp | Thread pointer | x18-x27 | s2-s11 | Saved registers |
| x5-x7 | t0-2 | Temporaries | x28-x31 | t3-t6 | Temporaries |

Assembler übersetzt zero = x0 usw.

# Zusammenfassung

- Sequentielle Befehlsbearbeitung
- Vorzeichenerweiterung
- Laden von Konstanten
- Weitere arithmetische/logische Befehle
- Übliche Speichereinteilung
- Funktion des Assemblers

# 2.2.1.5 Sprungbefehle

**Problem** mit dem bislang erklärten Befehlssatz: keine Abfragen möglich.

**Lösung**: (bedingte) Sprungbefehle (conditional branches).

Elementare RISC-V-Befehle:

branch if equal: **`beq x1,x2,label`**
mit **`x1, x2`** $\in$ **`x0...x31`**

branch if not equal: **`bne x1,x2,label`**
mit **`x1, x2`** $\in$ **`x0...x31`**

# Anwendung von beq und bne

**Übersetzung von**

```
if ( i == j ) goto L1
f = g + h;
L1: f = f - i;
```

**in**

```
      beq   x19, x20, L1       # nach L1, falls i=j
      add   x16, x17, x18      # f=g+h
L1:   sub   x16, x16, x19      # immer ausgeführt
```

# Anwendung von beq und bne

**Übersetzung von**

```
if ( i == j ) goto L1
f = g + h;
L1: f = f - i;
```

**in**

```
      beq   x19 , x20 , L1        # nach L1, falls i=j
      add   x16 , x17 , x18       # f=g+h
L1:   sub   x16, x16, x19         # immer ausgeführt
```

L1 ist die symbolische Bezeichnung für eine Befehlsadresse.

Der Assembler rechnet L1 in die im Maschinencode zu speichernde Konstante um.

# Unbedingte Sprünge

Problem: Übersetzung von

```
if ( i == j ) f = g + h ; else f = g - h ;
```

Lösung:

```
        bne   x19 , x20 , Else     # nach Else falls i≠j
        add   x16 , x17 , x18      # f=g+h
        j     Exit                 # Unbedingter Sprung
                                   # (unconditional jump)
Else:   sub   x16 , x17 , x18      # f=g-h
Exit:   ...
```

# Unbedingte Sprünge

Problem: Übersetzung von

```
if ( i == j ) f = g + h ; else f = g - h ;
```

Lösung:

```
        bne   x19 , x20 , Else      # nach Else falls i≠j
        add   x16 , x17 , x18       # f=g+h
        j     Exit                  # Unbedingter Sprung
                                    # (unconditional jump)
Else:   sub   x16 , x17 , x18       # f=g-h
Exit:   ...
```

# Realisierung von Array-Zugriffen

- Bei C beginnen Arrays mit Index 0
- Die Adresse des Arrays ist gleich der Adresse der Komponente 0 (z.B. = a_save)
- Wenn jede Array-Komponente ein Wort belegt, dann belegt Komponente mit Index $i$ das Wort $i$ des Arrays
- Sei c = Anzahl der adressierbaren Speicherzellen pro Element (=4 bei 32-Bit Integern auf RISC-V-Maschine),
  sei a_save = Array-Anfangsadresse, $\rightarrow$ (a_save+ $i$ x c) ist Adresse der Komponente $i$

# Realisierung von Schleifen mit unbedingten Sprüngen

**Problem**: Suche Element ‘k’ in array ‘save’

```
while ( save[ i] == k ) i = i + 1 ;
```

**Lösung**:

```
        li    x10, 4                # Reg[10] := 4
Loop:   mul   x9, x19 , x10         # Reg[9] := i * 4
        lw    x8, a_save(x9)        # Reg[8] := save[i]
        bne   x8, x21 , Exit        # nach Exit, falls ≠
        addi  x19, x19, 1           # i=i+1
        j     Loop                  # Pseudoinstruktion
Exit:
        ...
```

# Realisierung von Schleifen mit unbedingten Sprüngen

**Problem**: Suche Element ‘k’ in array ‘save’

```
while ( save[ i ] == k ) i = i + 1 ;
```

**Lösung**:

```
        li    x10, 4                # Reg[10] := 4
Loop:   mul   x9, x19 , x10         # Reg[9] := i * 4
        lw    x8, a_save(x9)        # Reg[8] := save[i]
        bne   x8, x21 , Exit        # nach Exit, falls ≠
        addi  x19, x19, 1           # i=i+1
        j     Loop                  # Pseudoinstruktion
Exit:
        ...
```

# Format und Bedeutung unbedingter Sprünge

**`j label`**

ist ein Pseudobefehl, welcher zum benannten Label springt.

**`li Konstant`**

ist ein Pseudobefehl, welcher ein (32-bit) Konstante lädt.

# Weitere Branch-Befehle

**Weitere Befehle:**

- `bge` (Verzweige für greater or equal)
- `blt` (Verzweige für less than)
- `bltz` (Verzweige für less than 0)
- `bgt` (Verzweige für greater than)
- `ble` (Verzweige für less or equal)
- `bgtz` (Verzweige für greater than 0)

# Tests auf <, ≤, >, ≥

RISC-V-Lösung: **`slt`**-Befehl (set if less than)

```
slt ra, rb, rc # Reg[ra] := if Reg[rb] < Reg[rc] then 1 else 0
```

Weitere Varianten:

- `slti` (Vergleich mit Direktoperanden)
- `sltu` (Vergleich für Betragszahlen)
- `sltiu` (Vergleich für Betragszahlen als Direktoperand)

# Übersetzung eine `for`-Schleife

Bedingte Sprünge werden auch zur Übersetzung von **`for`**-Schleifen benötigt.

Besipiel: Das C-Programm

```
j=0;
for ( i=0 ; i<n ; i++) j = j+i;
```

kann in das folgende Programm übersetzt werden:

```
main  li    x5, 0              # j:=0
      li    x6, 0              # i:=0
loop  bge   x6,x7, ende        # i>=n?
      add   x5,x5,x6           # j:=j+i
      addi  x6,x6,1            # i:=i+1
            loop
```

```
        j
ende    ...
```

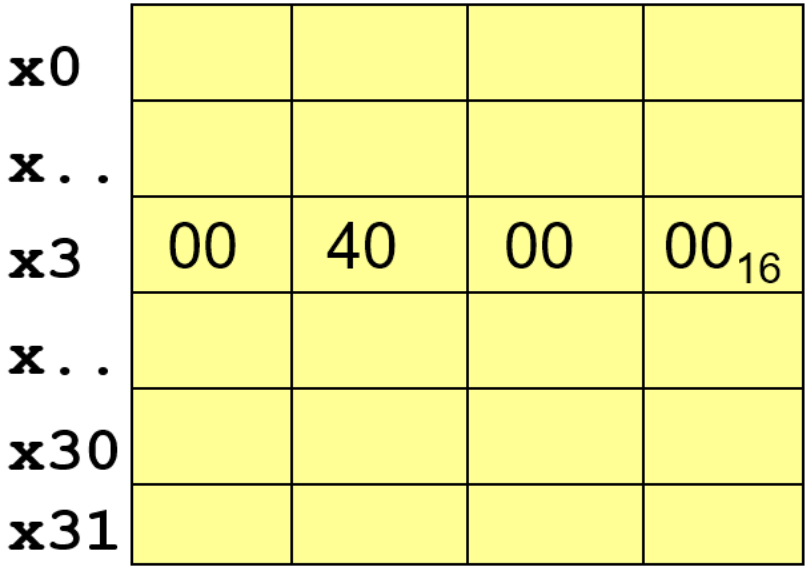
# Realisierung von berechneten Sprüngen: der `jr`-Befehl

Format: **`jr reg`** (jump register), mit **`reg`**$\in$ **`x0...x31`**

Semantik: **`PC:=Reg[reg]`**

Beispiel: **`jr x3`**

Registerspeicher (**`Reg`**)




PC

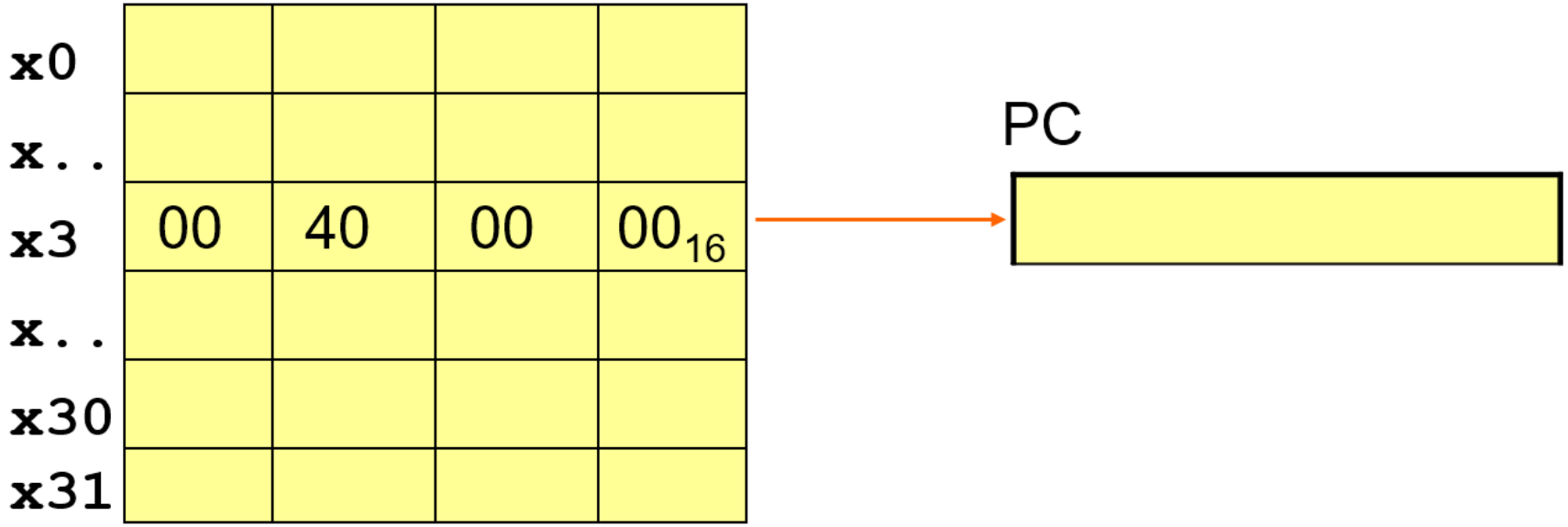
# Realisierung von berechneten Sprüngen: der `jr`-Befehl

Format: **`jr reg`** (jump register), mit **`reg`**$\in$ **`x0...x31`**

Semantik: **`PC:=Reg[reg]`**

Beispiel: **`jr x3`**

Registerspeicher (**`Reg`**)


x0
x..
x3
00
40
00
$00_{16}$
x..
x30
x31
PC

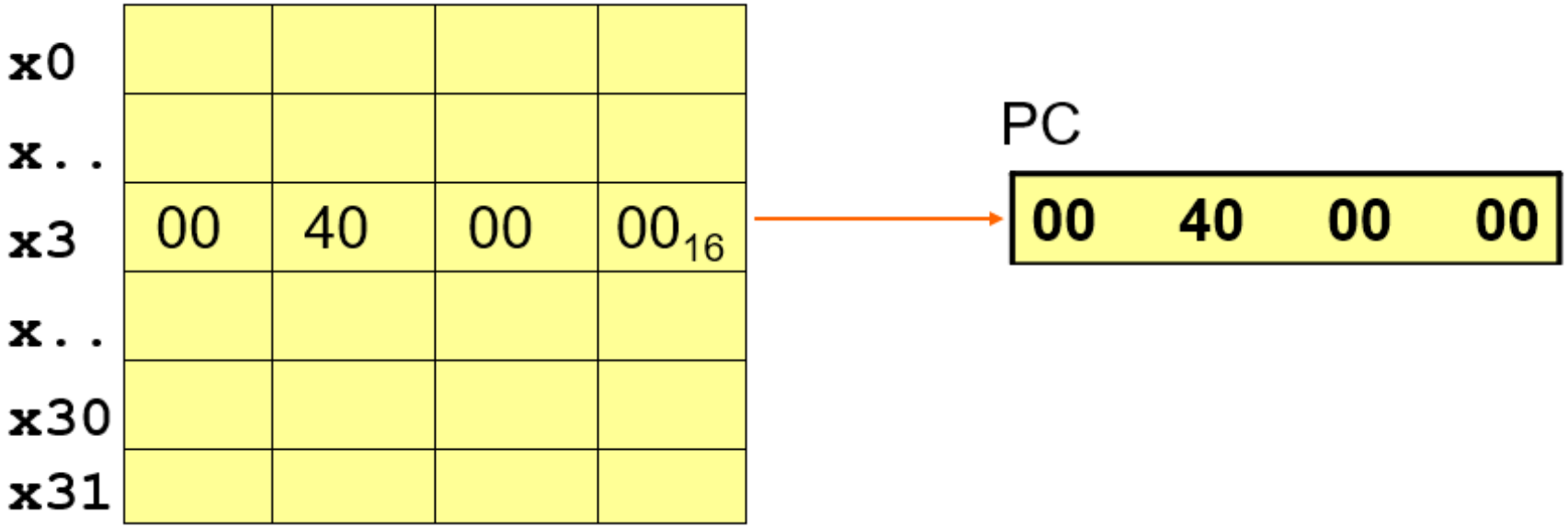
# Realisierung von berechneten Sprüngen: der `jr`-Befehl

Format: **`jr reg`** (jump register), mit **`reg`**$\in$ **`x0...x31`**

Semantik: **`PC:=Reg[reg]`**

Beispiel: **`jr x3`**

Registerspeicher (**`Reg`**)


x0
x..
x3
00
40
00
00₁₆
x..
x30
x31
PC
00
40
00
00

# Switch-Anweisungen

Annahme: k=2

```
switch(k) { /* k muss ∈ 0...3 sein! */
case 0: f=i+j; break; /* k=0 */
case 1: f=g+h; break; /* k=1 */
case 2: f=g-h; break; /* k=2 */
case 3: f=i-j; break; /* k=3 */
}
```

Adresse des ersten Befehls für case 0

Adresse des ersten Befehls für case 3

```
.data
Jumptable: .word L0, L1, L2, L3, ...
```

**Realisierung Jumptable**

| ... |
|---|
| L0 |
| L1 |
| L2 |
| L3 |
| ... |

# Realisierung von SWITCH-Anweisungen mittels des jr-Befehls

```
switch(k) {
  case 0: f=i+j; break; /* k=0 */
  case 1: f=g+h; break; /* k=1 */
  case 2: f=g-h; break; /* k=2 */
  case 3: f=i-j; break; /* k=3 */
}
```

```
        li  x10, 4              # Reg[10] := 4
swit:   mul x9,  x10, x21       # Reg[9] :=k*4
        lw  x8,  Jumptable(x9) # 1 der 4 Adressen
        jr  x8                  # PC:=Reg[8]
L0:     add x16, x19, x20       # k ist 0;
        j   Exit                # entspircht break
L1:     add x16, x17, x18       # k ist 1;
        j   Exit                # entspircht break
L2:     sub x16, x17, x18       # k ist 2;
        j   Exit                # entspircht break
L3:     sub x16, x19, x20       # k ist 3;
Exit:   . . .                   #
```

# Zusammenfassung

- Sprungbefehle
  - Bedingte Sprungbefehle
  - Unbedingte Sprungbefehle
- Realisierung von
  - `if`-Anweisungen
  - `while`- und `for`-Schleifen
  - `switch`-Anweisungen

# 2.2.1.6 Prozeduraufrufe

```
void function C
{
  ...
}
void function B
{
C
}
void function A
{
B
```

Man muss

- sich merken können, welches der auf den aktuellen Befehl im Speicher folgende ist (d.h., man muss sich PC+4 merken) und
- an eine (Befehls-)Adresse springen

```
}
main: A
```

# Der Jump-and-Link Befehl

**`jal rd, imm # sign_ext(imm(20:1) & "0", 32) + PC = Zieladresse`**

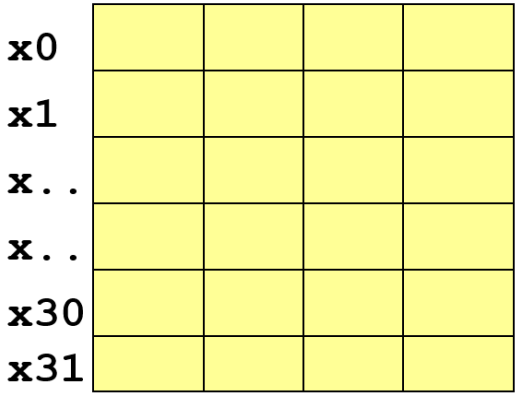
**`imm[20:1]`** generiert eine Sprungweite von $2^{21} = 2$ MB relativ zum **`PC`**

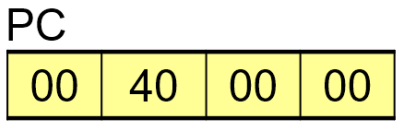
Der **`j`** Pseudobefhel wird mit **`jal`** realisiert

RISC-V-Konvention: Benutzung von Register 1 als *return-address*

| | **Größe [Bit]** | **20** | **5** | **7** |
|---|---|---|---|---|
| `jal, j` | Format UJ | imm(20:1) | rd | op |

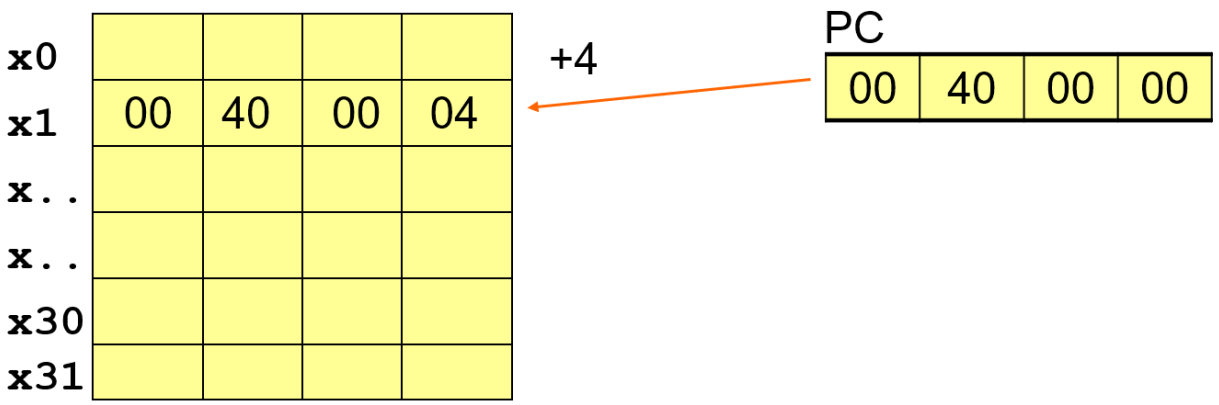
# Realisierung nicht verschachtelter Prozeduraufrufe mit `jal` und `jr`



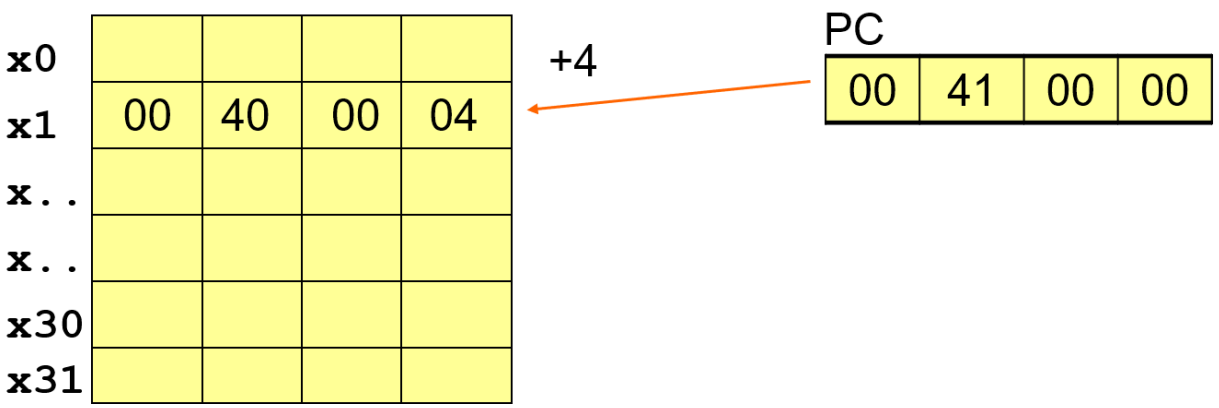

```
        void function A
        {
410000  ...
410004          /* jr, x1 */
        }
400000  main: A; /*jalr A*/
400004  ...
```

# Realisierung nicht verschachtelter Prozeduraufrufe mit `jal` und `jr`



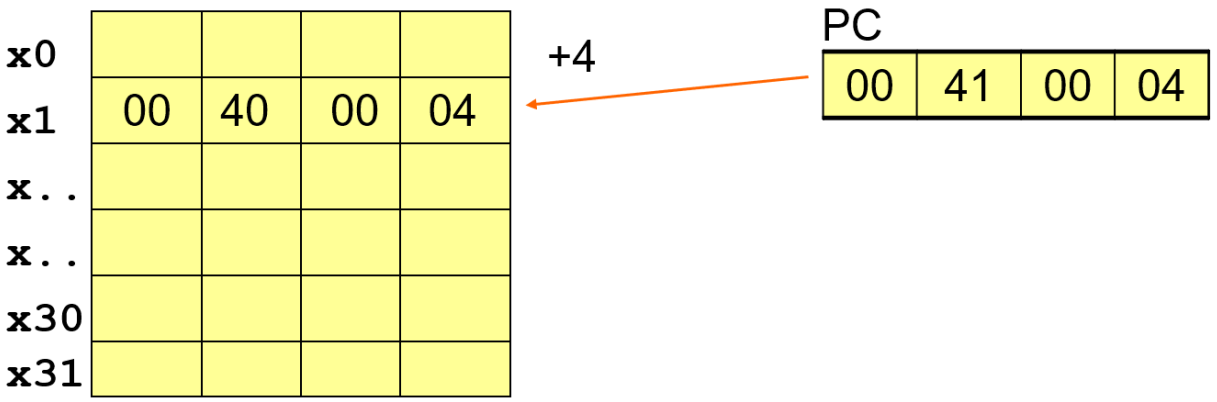

```
        void function A
        {
410000  ...
410004          /* jr, x1 */
        }
400000 main: A; /*jalr A*/
400004 ...
```

# Realisierung nicht verschachtelter Prozeduraufrufe mit `jal` und `jr`



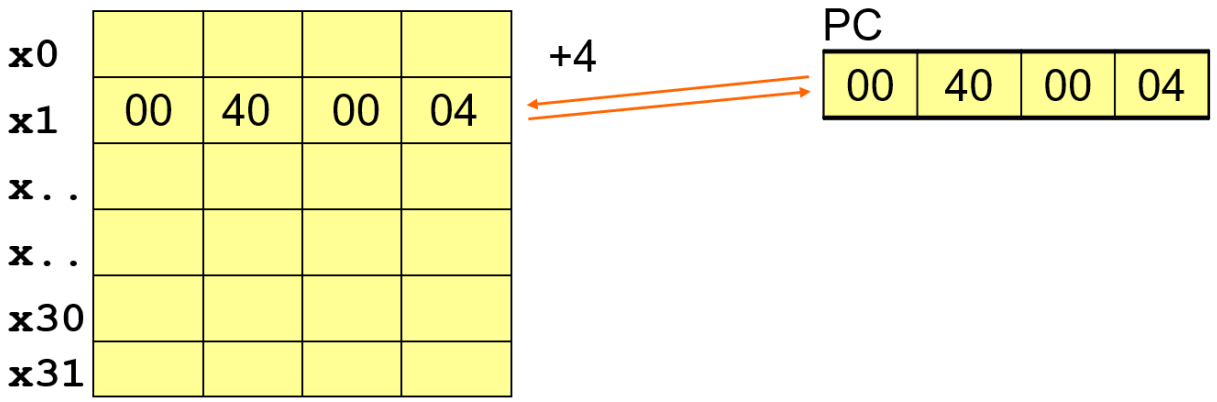

```
        void function A
        {
410000  ...
410004          /* jr, x1 */
        }
400000 main: A; /*jalr A*/
400004 ...
```

# Realisierung nicht verschachtelter Prozeduraufrufe mit `jal` und `jr`




```
        void function A
        {
410000  ...
410004          /* jr, x1 */
        }
400000 main: A; /*jalr A*/
400004 ...
```

# Realisierung nicht verschachtelter Prozeduraufrufe mit `jal` und `jr`




```
        void function A
        {
410000  ...
410004          /* jr, x1 */
        }
400000 main: A; /*jalr A*/
400004 ...
```

**ABER:**

Sprünge außerhalb von 2MB (±1MB) lassen sich so nicht darstellen!

⇒ Ein anderer Befehl ist nötig

# Der *jump-and-link-register*-Befehl (1)

Format: **`jalr rd, rs, imm`**
wobei **`imm`** ein Offset von $2^{12}$ ist ($2^{11} - 1$ bis $-2^{11}$)

Bedeutung: **`Reg[rd] := PC+4; # rd ist normalerweise x1`**
**`PC := (Reg[rs]+sign_ext(imm(11:0), 32)) (31:1) & "0";`**
Das letzte Bit der Immediate ist immer 0, um auf Halbwortlänge springen zu können

Beispiel: **`jalr x1, x2, 0x0f8 # x2:=0x0fff000`**

| Registerspeicher (Reg) | | | | |
|---|---|---|---|---|
| x0 | | | | |
| x1 | | | | |
| x.. | | | | |
| x.. | | | | |
| x31 | | | | |

PC | 00 | 40 | 00 | 00 |

„The JALR instruction ignores the lowest bit of the calculated target address.“ [The RISC-V Instruction Set Manual Volume I: User-Level ISA]

# Der *jump-and-link-register*-Befehl (1)

Format: **`jalr rd, rs, imm`**
wobei **`imm`** ein Offset von $2^{12}$ ist ($2^{11} - 1$ bis $-2^{11}$)

Bedeutung: **`Reg[rd] := PC+4; # rd ist normalerweise x1`**
**`PC := (Reg[rs]+sign_ext(imm(11:0), 32)) (31:1) & "0";`**
Das letzte Bit der Immediate ist immer 0, um auf Halbwortlänge springen zu können

Beispiel: **`jalr x1, x2, 0x0f8 # x2:=0x0fff000`**

| Registerspeicher (Reg) | | | | |
|---|---|---|---|---|
| x0 | | | | |
| x1 | 00 | 40 | 00 | 04 |
| x.. | | | | |
| x.. | | | | |
| x31 | | | | |

Nach der Ausführung von **jalr** an Adresse $400000_{16}$

PC | 0x 0f ff f0 f8

„The JALR instruction ignores the lowest bit of the calculated target address.“ [The RISC-V Instruction Set Manual Volume I: User-Level ISA]

# Der *jump-and-link-register*-Befehl (2)

| | Größe [Bit] | 12 | 5 | 3 | 5 | 7 |
|---|---|---|---|---|---|---|
| `jalr` | Format I | imm(11:0) | rs1 | funct3 | rd | op |

Lade in **`rs1`** eine Upper-Immediate (**`lui`**) und führe dann einen Sprung aus. Damit wird eine Sprungweite über die ganzen 32 Bits erreicht.

```
lui x2, 0x0ffff     # Bereite Sprung vor
jalr x1, 0x7f4(x2)  # Führe Sprung zu 0x0fff7f4durch
```

# Das Stapel-Prinzip

```
void function C
{
  ...
}
void function B
{
C
}
void function A
{
B
```

Speicherbereich für (1. Aufruf) von C

Speicherbereich für (1. Aufruf) von B

Speicherbereich für (1. Aufruf) von A

Evtl. Speicherbereich für Hauptprogramm

```
}
main: A
```

# Das Stapel-Prinzip

```
void function C
{
  ...
}
void function B
{
C
}
void function A
{
B
```

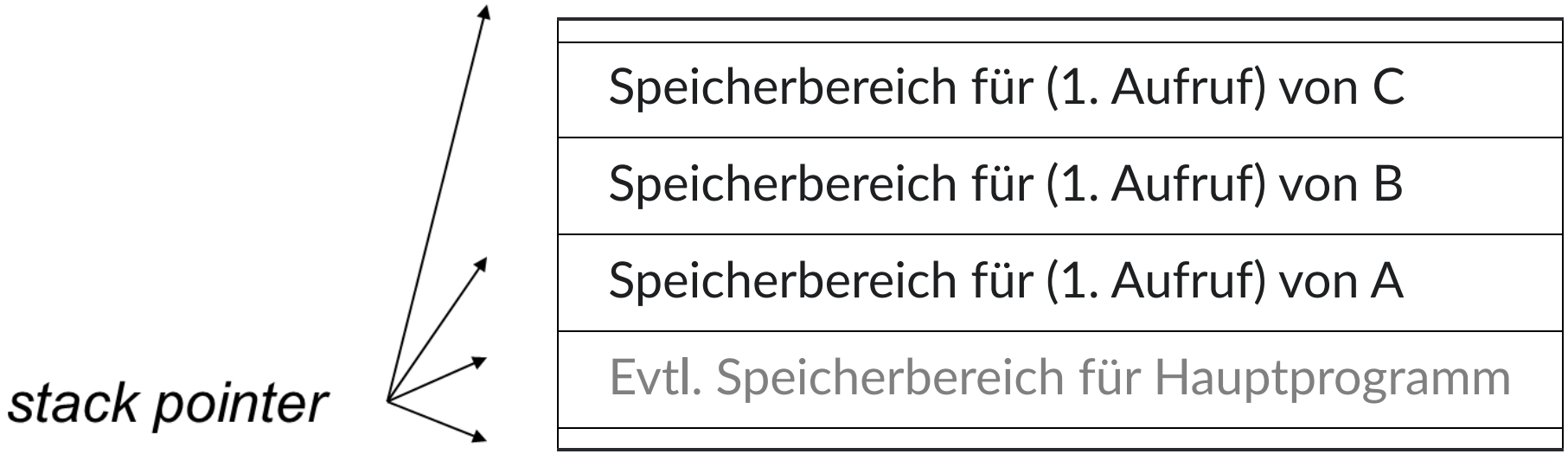
Speicherbereich für (1. Aufruf) von B

Speicherbereich für (1. Aufruf) von A

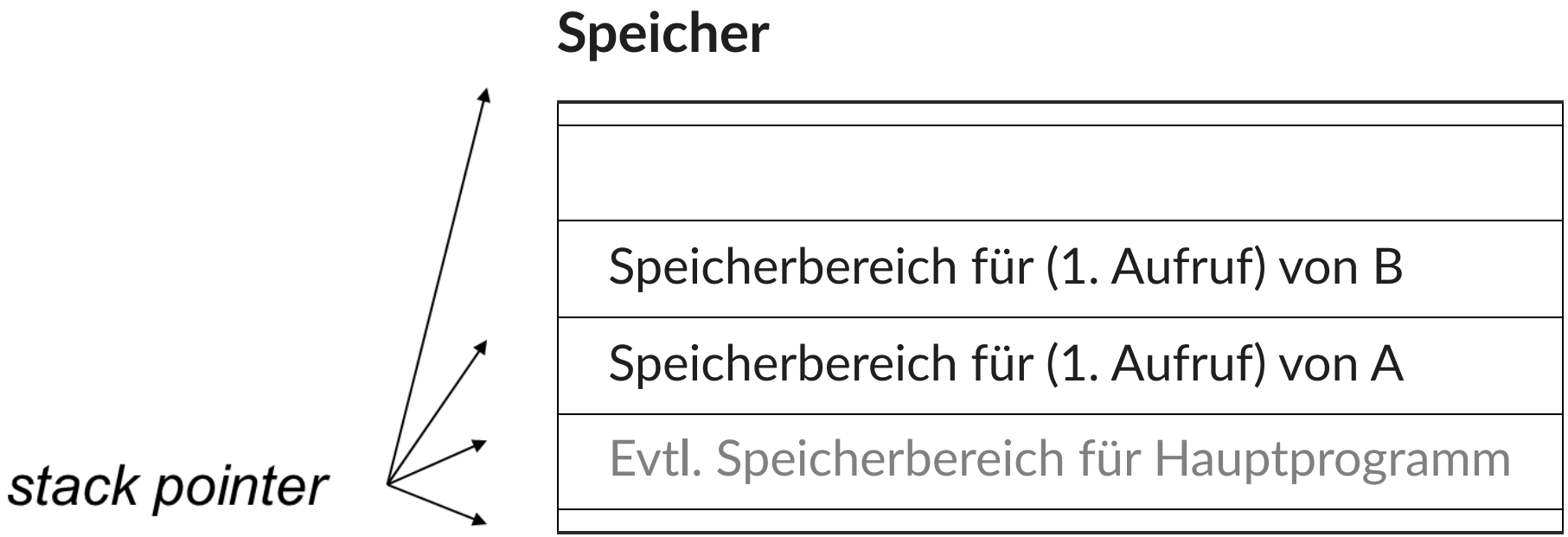
Evtl. Speicherbereich für Hauptprogramm

```
}
main: A
```

# Realisierung eines Stapels im Speicher



# Realisierung eines Stapels im Speicher




RISC-V-Konvention: Benutzung von Register 2 als stack-pointer

# Konkrete Realisierung für drei Prozeduren

**Statisch:**

```
C()
    { ...; }    /* jr /*
B()
    {           /*x1->stack*/
        C();
    }           /*stack->x1, jr */
A()
    {           /*x1->stack*/
        B();
    }           /*stack->x1, jr */
main()
    {A();}
```

**Dynamisch:**

| Aktiv | Befehl |
| --- | --- |
| main | jal x1, A |
| A | x1 -> stack |
| A | jal x1, B |
| B | x1 -> stack |
| B | jal x1, C |
| C | jr x1 |
| B | Rückschreiben von x1 |
| B | jr x1 |
| A | Rückschreiben von x1 |
| A | jr x1 |
| main | |

# Sichern von Registerinhalten

Einige Register sollten von allen Prozeduren genutzt werden können, unabhängig davon, welche Prozeduren sie rufen und von welchen Prozeduren sie gerufen werden.

Die Registerinhalte müssen beim Prozeduraufruf gerettet und nach dem Aufruf zurückgeschrieben werden.

Zwei Methoden:

1. **caller save**: Aufrufende Prozedur rettet vor Unterprogrammaufruf Registerinhalte und kopiert sie danach zurück.
2. **callee save**: Gerufene Prozedur rettet nach dem Unterprogramm-Aufruf diese Registerinhalte und kopiert sie vor dem Rücksprung zurück.

# Sichern von Registern (2)

**caller save**

caller:
Retten der Register auf den *stack*
`jalr...`

callee:
Retten von `x1`.
Befehle für Rumpf.
Rückschreiben von `x1`
`jr x1`

caller:
Rückschreiben der Register.

**callee save**

caller:
`jalr...`

callee:
Retten der Register auf den *stack*
Retten von `x1`.
Befehle für Rumpf.
Rückschreiben von `x1`. Rückschreiben der Register.
`jr x1`.

caller:
...

# 2.2.1.7 Prozeduren mit Parametern

```
int stund2sec(int stund) {
return stund*60*60;
}
stund2sec(5);
```

Wo findet **`stund2sec`** den Eingabeparameter?

**Konflikt:**

- Parameter möglichst in Registern übergeben (schnell)
- Man muss eine beliebige Anzahl von Parametern erlauben

**RISC-V-Konvention:** Die ersten acht Parameter werden in den Registern `x10-17` übergeben, alle weiteren im *stack*.

# Benutzung der Register

| Register | Bedeutung | Register | Bedeutung |
|---|---|---|---|
| x0 | Always 0 | x8 | Saved register/ frame pointer |
| x1 | Return address | x9 | Saved register |
| x2 | Stack pointer | **x10-x11** | **Function arguments/ return values** |
| x3 | Global pointer | **x12-x17** | **Function arguments** |
| x4 | Thread pointer | x18-x27 | Saved registers |
| x5-x7 | Temporaries | x28-x31 | Temporaries |

# Call by value vs. Call by reference

## Parameterwert an Prozeduren oder dessen Adresse übergeben?

### call by value

- Parameterwert selbst
- gerufener Prozedur ist nur der Wert bekannt
- Speicherstelle, an welcher er gespeichert ist, kann nicht verändert werden

### call by reference

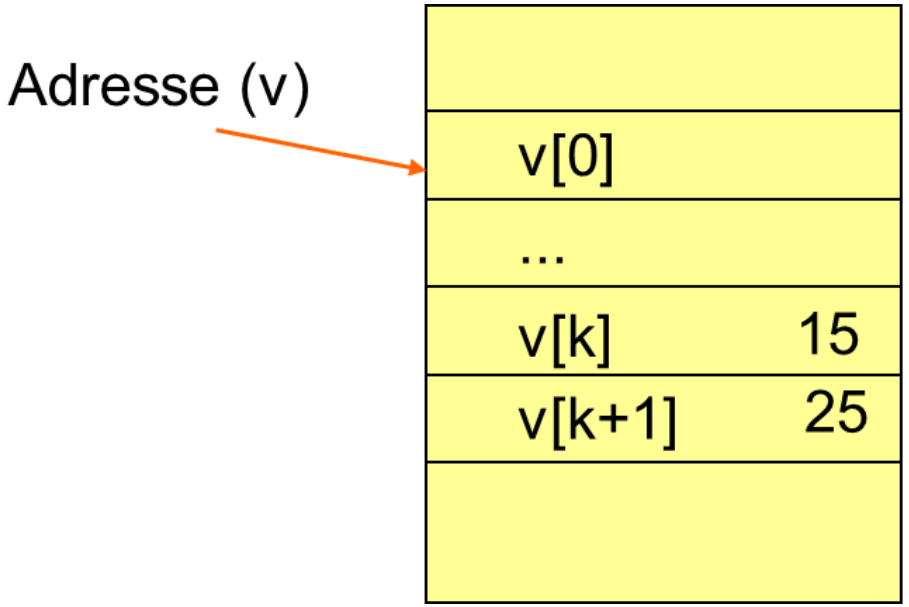
- Adresse des Parameters
- Erlaubt Ausgabeparameter
- Änderung der Werte im Speicher durch die gerufene Prozedur möglich
- Bei großen Strukturen (*arrays*) effizient

## In C:

- Bei skalaren Datentypen (*int*, Zeigern, usw.): wählbar, typischerweise *call by value*
- Bei komplexen Datentypen (*arrays*):
  *call by reference*

# Prozedur swap,
# Prinzip der Übersetzung in Assembler

```
swap (int v[], int k)
{ int temp;
temp=v[k];
v[k]=v[k+1];
v[k+1]=temp;
}
```




**Schritte der Übersetzung in Assembler:**

1. Zuordnung von Speicherzellen zu Variablen
2. Übersetzung des Prozedurrumpfes in Assembler

3. Sichern und Rückschreiben der Register

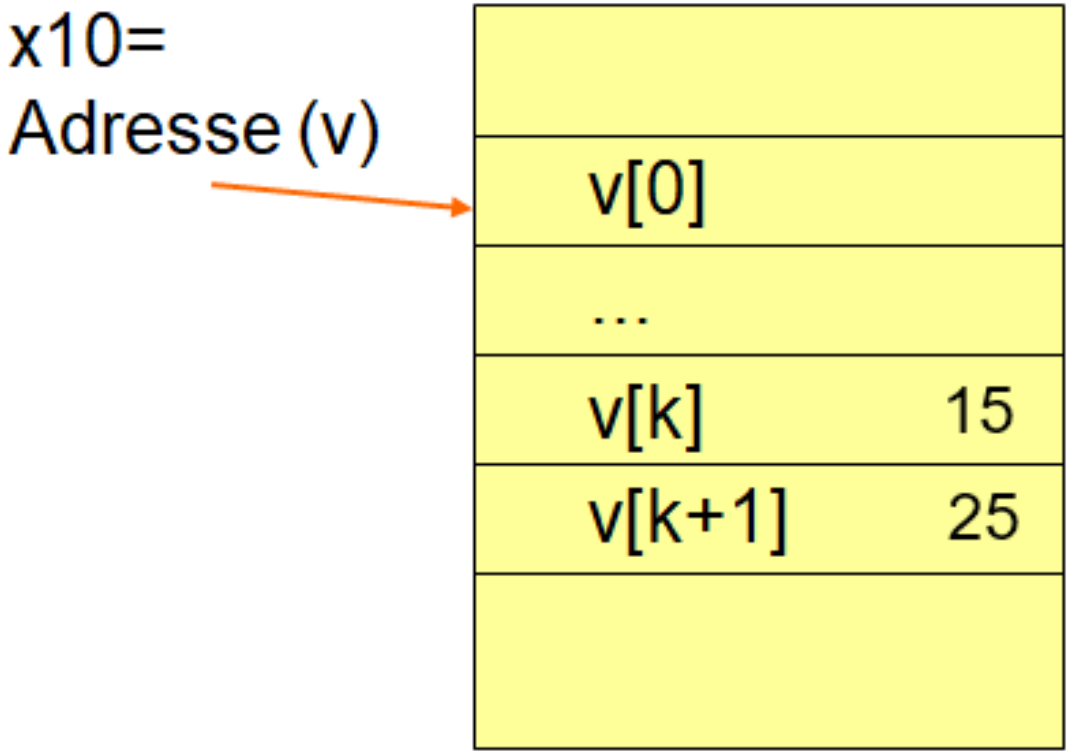
# Zuordnung von Registern

| Variable | Speicherzelle | Kommentar |
|---|---|---|
| Adresse von v | `x10` | Aufgrund der RISC-V-Konvention für den 1. Parameter |
| k | x11 | Aufgrund der RISC-V-Konvention für den 2. Parameter |
| temp | x15 | (irgendein freies Register) |
| Interne Hilfsvariable | x16 | (irgendein freies Register) |

# Realisierung des Rumpfes

```
    temp=v[k];
    v[k]=v[k+1];
    v[k+1]=temp
```

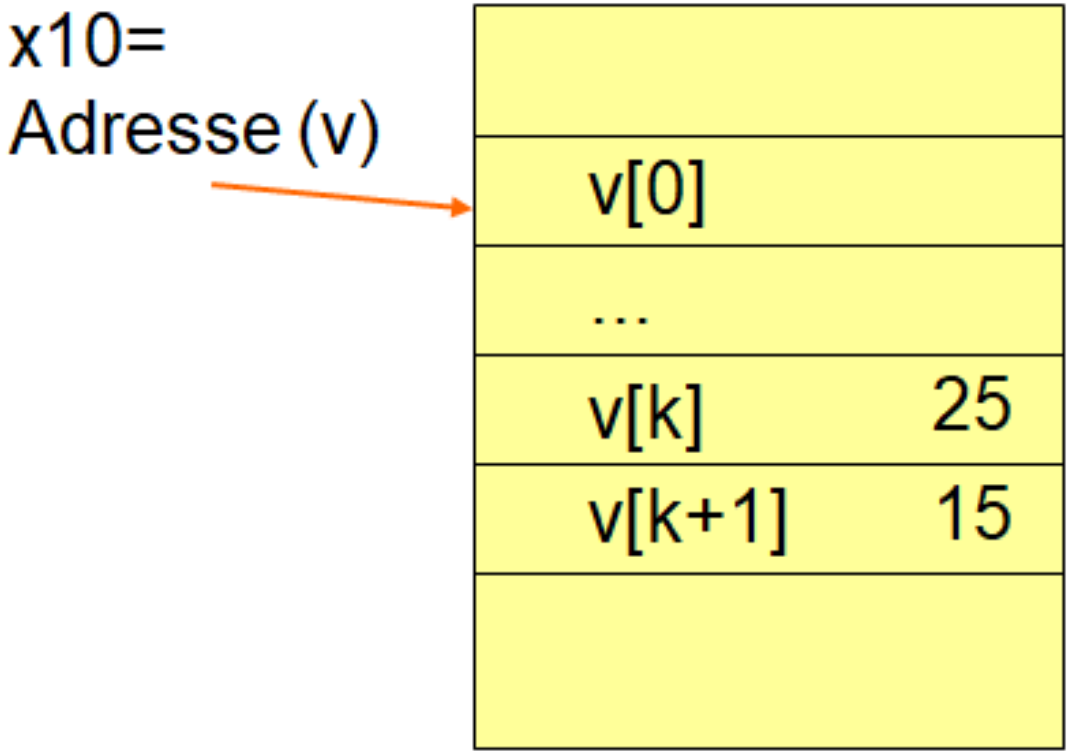
```
li  x5,  4       #
mul x5,  x5, x11 #Reg[5]:=4*k
add x5,  x10, x5 #Adresse von v[k]
lw  x15, 0(x5)   #lade v[k]
lw  x16, 4(x5)   #lade v[k+1]
sw  x16, 0(x5)   #v[k]:=v[k+1]
sw  x15, 4(x5)   #v[k+1]:=temp
```



# Realisierung des Rumpfes

```
    temp=v[k];
    v[k]=v[k+1];
    v[k+1]=temp
```

```
li  x5,  4        #
mul x5,  x5, x11 #Reg[5]:=4*k
add x5,  x10, x5 #Adresse von v[k]
lw  x15, 0(x5)   #lade v[k]
lw  x16, 4(x5)   #lade v[k+1]
sw  x16, 0(x5)   #v[k]:=v[k+1]
sw  x15, 4(x5)   #v[k+1]:=temp
```



# Sichern der Register

**Prolog: Sichern der in swap überschriebenen Register**

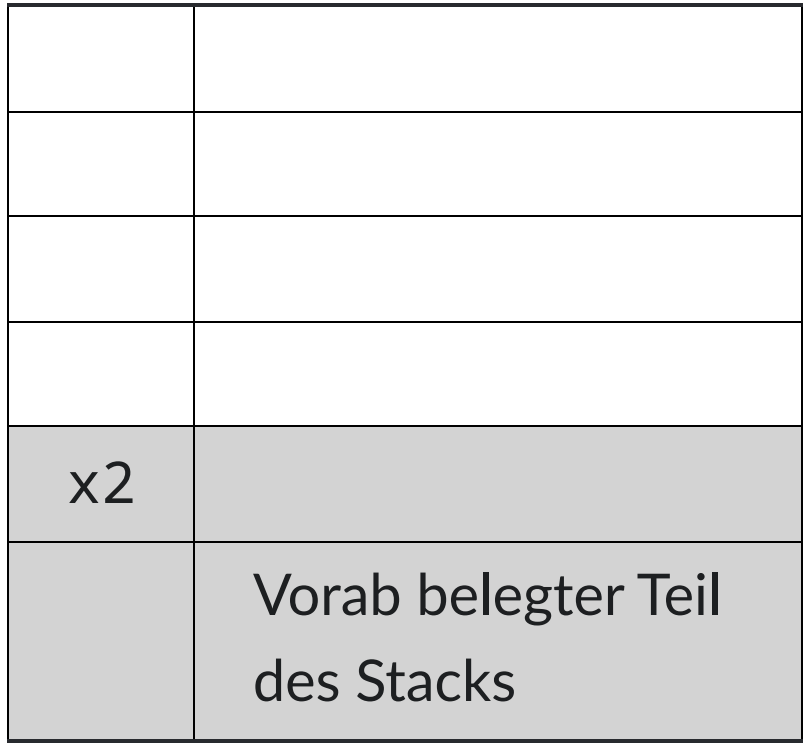
**Prolog**

Prozedurrumpf

Epilog

```
addi x2, x2, -16
sw x5, 0(x2)
sw x15, 4(x2)
sw x16, 8(x2)
sw x1, 12(x2)
```

Speicher




Wegen der Verwendung des Stacks auch für Interrupts (siehe 2.2.2.4) immer **zuerst**

mittels `addi` den Platz auf dem Stack schaffen!

# Sichern der Register

**Prolog: Sichern der in swap überschriebenen Register**

**Prolog**

Prozedurrumpf

Epilog

```
addi x2, x2, -16
sw x5, 0(x2)
sw x15, 4(x2)
sw x16, 8(x2)
sw x1, 12(x2)
```

Speicher

| | |
|---|---|
| x2 | |
| | |
| | |
| | |
| | |
| | Vorab belegter Teil des Stacks |

Wegen der Verwendung des Stacks auch für Interrupts (siehe 2.2.2.4) immer **zuerst**

mittels `addi` den Platz auf dem Stack schaffen!

# Sichern der Register

**Prolog: Sichern der in swap überschriebenen Register**

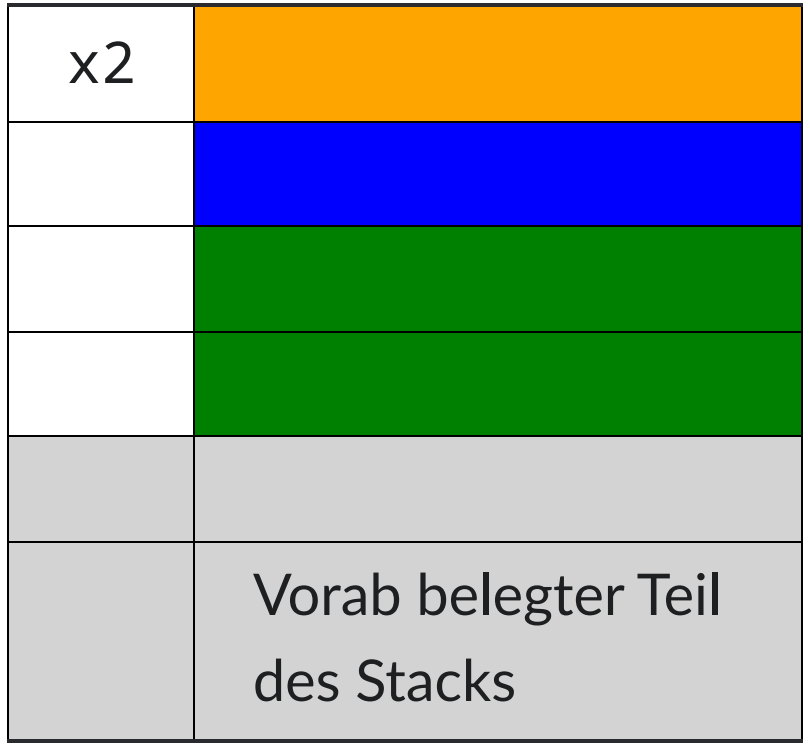
**Prolog**

Prozedurrumpf

Epilog

```
addi x2, x2, -16
sw x5, 0(x2)
sw x15, 4(x2)
sw x16, 8(x2)
sw x1, 12(x2)
```

Speicher




Wegen der Verwendung des Stacks auch für Interrupts (siehe 2.2.2.4) immer **zuerst**

mittels `addi` den Platz auf dem Stack schaffen!

# Sichern der Register

**Prolog: Sichern der in swap überschriebenen Register:**

**Prolog**

Prozedurrumpf

Epilog

```
addi x2, x2, -16
sw x5, 0(x2)
sw x15, 4(x2)
sw x16, 8(x2)
sw x1, 12(x2)
```

Speicher

| | |
|---|---|
| x2 | Nach dem Sichern belegter Teil des Stacks |
| | |
| | Vorab belegter Teil des Stacks |

Wegen der Verwendung des Stacks auch für Interrupts (siehe 2.2.2.4) immer **zuerst** mittels `addi` den Platz auf dem Stack schaffen!

# Rückschreiben der Register

**Epilog: Rückschreiben der in swap überschriebenen Register:**

Prolog

Prozedurrumpf

**Epilog**

```
lw x5, 0(x2)
lw x15, 4(x2)
lw x16, 8(x2)
lw x1, 12(x2)
addi x2, x2, 16
```

Speicher

| | |
|---|---|
| x2 | Nach dem Sichern belegter Teil des Stacks |
| | |
| | Vorab belegter Teil des Stacks |

Wegen der Verwendung des Stacks auch für Interrupts (siehe 2.2.2.3) immer **zuletzt** mittels `addi` den Platz auf dem Stack freigeben!

# Rückschreiben der Register

**Epilog: Rückschreiben der in swap überschriebenen Register:**

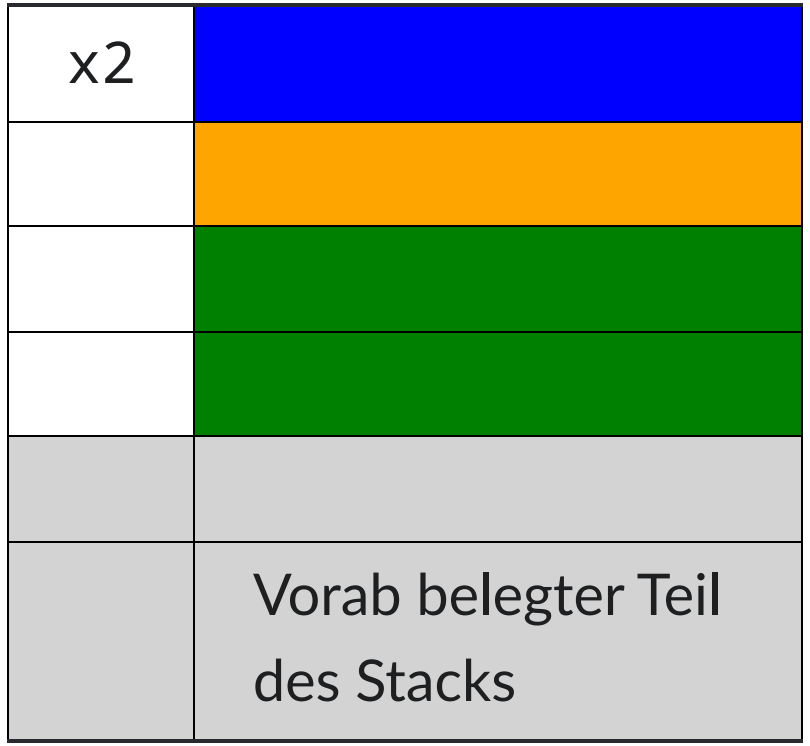
Prolog

Prozedurrumpf

**Epilog**

```
lw x5, 0(x2)
lw x15, 4(x2)
lw x16, 8(x2)
lw x1, 12(x2)
addi x2, x2, 16
```

Speicher




Wegen der Verwendung des Stacks auch für Interrupts (siehe 2.2.2.3) immer **zuletzt**

mittels `addi` den Platz auf dem Stack freigeben!

# Rückschreiben der Register

**Epilog: Rückschreiben der in swap überschriebenen Register:**

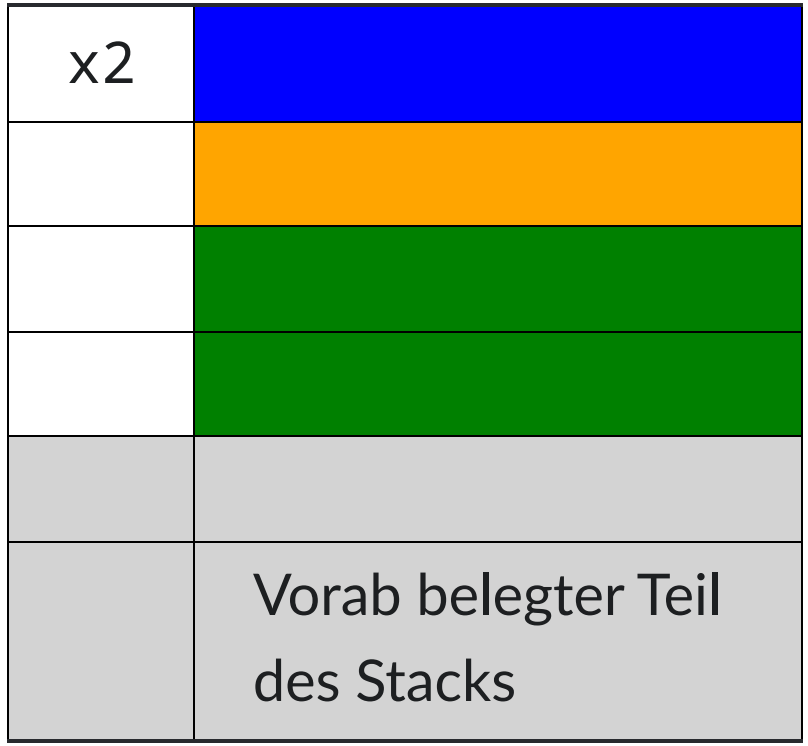
Prolog

Prozedurrumpf

**Epilog**

```
lw x5, 0(x2)
lw x15, 4(x2)
lw x16, 8(x2)
lw x1, 12(x2)
addi x2, x2, 16
```

Speicher




Wegen der Verwendung des Stacks auch für Interrupts (siehe 2.2.2.3) immer **zuletzt**

mittels `addi` den Platz auf dem Stack freigeben!

# Rückschreiben der Register

**Epilog: Rückschreiben der in swap überschriebenen Register:**

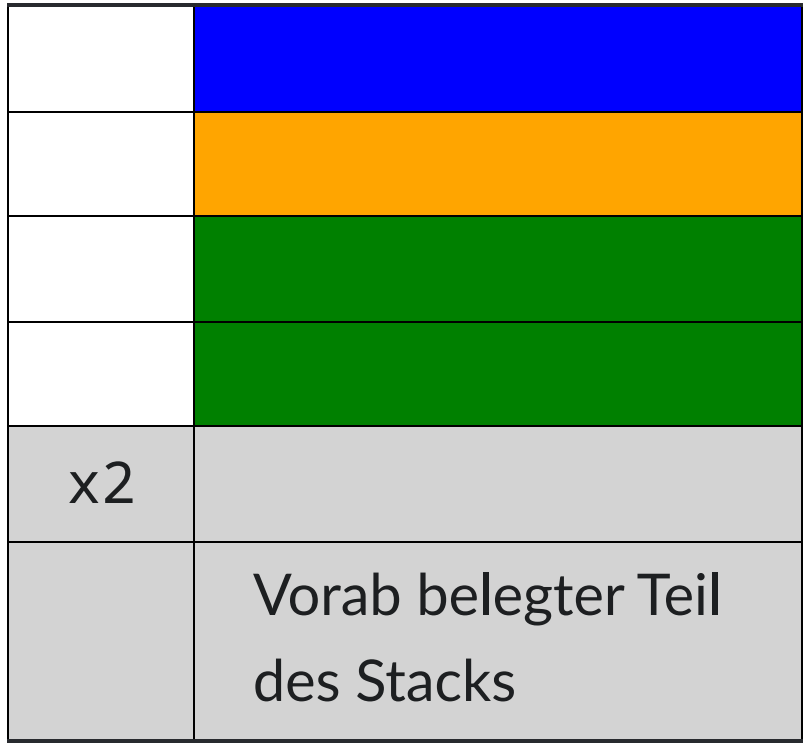
Prolog

Prozedurrumpf

**Epilog**

```
lw x5, 0(x2)
lw x15, 4(x2)
lw x16, 8(x2)
lw x1, 12(x2)
addi x2, x2, 16
```

Speicher




Wegen der Verwendung des Stacks auch für Interrupts (siehe 2.2.2.3) immer **zuletzt**

mittels `addi` den Platz auf dem Stack freigeben!

# Übersetzung von insertion sort

```
int v[10000]
sort (int v[], int n)
{ int, j;
    for (i=0; i<n; i=i+1) {
        for (j=i-1; j>=0 && v[j]>v[j+1]; j=j-1) {
            swap(v, j);
        }
    }
}
```

**Schritte der Übersetzung in Assembler:**

1. Zuordnung von Speicherzellen zu Variablen
2. Übersetzung des Prozedurrumpfes in Assembler
3. Sichern und Rückschreiben der Registerinhalte

# Zuordnung von Registern

| | | |
|---|---|---|
| Adresse von v | `x10` | Lt. Konvention |
| n | `x11` | Lt. Konvention |
| j | x22 | gesichertes Register |
| Kopie von `x10` | `x19` | Sichern der Parameter zur Vorbereitung des Aufrufs von swap |
| i | `x21` | gesichertes Register |
| Kopie von `x11` | `x22` | wie `x10` |
| Hilfsvariable | `x28-31` | irgendwelche freien Register |
| Hilfsvariable | `x23` | nicht gesichert |

# Übersetzung des Rumpfes

```
int v[10000]
sort (int v[], int n)
{ int, j;
    for (i=0; i<n; i=i+1) {
        for (j=i-1; j>=0 && v[j]>v[j+1]; j=j-1) {
            swap(v, j);
        }
    }
}
```

| | |
|---|---|
| Adr. von v | x10 |
| n | x11 |
| j | x22 |
| Kopie von x10 | x19 |
| i | x21 |
| Kopie von x11 | x20 |
| Hilfsvariable | x28-31 |
| Hilfsvariable, nicht gesichert | x23 |

```
      add  x19, x10, x0   #
      add  x20, x11, x0   #
      li   x21, 0         #i:=0
for1: bge  x21, x20, ex1  #i>=n?
      addi x22, x21, -1   #j:=i-1
for2: slti x23, x22, 0    #j<0?
      bne  x23, x0,  ex2  #
      li   x23, 4         #x23:=4
      mul  x28, x22, x23  #j*4
      add  x29, x10, x28  #Adr(v[j])
      lw   x30, 0(x29)    #v[j]
      lw   x31, 4(x29)    #v[j+1]
      ble  x30, x31, ex2  #v[j]<=j[j+1]
      add  x10, x19, x0   #1.Param
      add  x11, x22, x0   #2.Param
      jal  x1,  swap      #
      addi x22, x22, -1   #j:j-1
      j    for2           #for-ende
ex2:  addi x21, x21, 1    #i:=i+1
      j    for1           #for-ende
ex1:  li   x17,10
      ecall
```

# Sichern der Registerinhalte

**Prolog: Sichern der überschriebenen Register**

**Prolog**

Prozedurrumpf

Epilog

| | |
|---|---|
| Adresse von v | x10 |
| n | x11 |
| j | x22 |
| Kopie von x10 | x19 |
| i | x21 |
| Kopie von x11 | x20 |
| Hilfsvariable | x28-31 |
| Hilfsvariable, nicht gesichert | x23 |

```
addi x2, x2,-36
sw x19, 0(x2)
sw x20, 4(x2)
sw x21, 8(x2)
sw x22, 12(x2)
sw x28, 16(x2)
sw x29, 20(x2)
sw x30, 24(x2)
sw x31, 28(x2)
sw x1, 32(x2)
```

# Rückschreiben der Registerinhalte

**Epilog: Rückschreiben der Registerinhalte**

Prolog

Prozedurrumpf

**Epilog**

| | |
|---|---|
| Adresse von v | `x10` |
| n | `x11` |
| j | `x22` |
| Kopie von `x10` | `x19` |
| i | `x21` |
| Kopie von `x11` | `x20` |
| Hilfsvariable | `x28-31` |
| Hilfsvariable, nicht gesichert | `x23` |

```
lw x19, 0(x2)
lw x20, 4(x2)
lw x21, 8(x2)
lw x22, 12(x2)
lw x28, 16(x2)
lw x29, 20(x2)
lw x30, 24(x2)
lw x31, 28(x2)
lw x1, 32(x2)
addi x2, x2, 36
jr x1
```

# Zusammenfassung

- Nicht-verschachtelte Prozeduren
- `jr`- und `jalr`-Befehle
- Das Stapel- (*stack*) Prinzip
- Verschachtelte Prozeduren
- Konventionen zu Registernutzung
- *Call-by-value*, *call-by-reference*
- Sichern und Rückschreiben von Registerinhalten
- *Callee-save* und *caller-save*

- Beispiele

# 2.2. Allgemeine Sicht der Befehlsschnittstelle

Bislang haben wir ein spezielles Beispiel einer Befehlsschnittstelle betrachtet.

→ Jetzt allgemeine Sicht auf die Befehlsschnittstelle

Zu Befehlsschnittstelle (**instruction set architecture, ISA**) gehören:

1. Die nutzbaren **Maschinenbefehle** (einschl. der elementaren Datentypen, siehe 2.2.2.2)
2. Ein **Modell des Speichers** (siehe 2.2.2.1)
3. Die logische Sicht auf das **Unterbrechungssystem** (*interrupt system*, siehe 2.2.2.3)

# Vorlesung Rechnerstrukturen, Teil 2

# 2 Rechnerarchitektur

# 2.2 Die Befehlsschnittstelle

# 2.2.2 Allgemeine Sicht der Befehlsschnittstelle

# 2.2.2.1 Das Speichermodell

# Das Speichermodell

- **Zeitkomplexität des Zugriffs**: Verhalten entspricht dem eines großen Arrays, direkter Zugriff (ohne sequent. Suche)
- Größe der **adressierbaren Einheiten**: meist Byte-adressierbar
- **Alignment**-Beschränkungen beschreiben Einschränkungen hinsichtlich der Ausrichtung (z.B.: Befehle und *int's* beginnen auf Wortgrenzen)
- Maximale **Größe** des Speichers: Historisch gesehen immer wieder unterschätzt.
  "640k will be enough for everyone" - Bill Gates (?), 1981
  → Verdopplung alle 2-3 Jahre
- Die Speicher realisieren den **Zustand** des Rechnersystems

# *Little endians* und *big endians* (1) (Endianess)

Ursprung: Gulliver’s Reisen: Ei am dicken oder dünnen Ende aufschlagen?

**Little endians**: der am wenigsten signifikate Teil eines Wortes erhält die niedrigste Byteadresse

**Big endians**: der signifikanteste Teil eines Wortes erhält die niedrigste Byteadresse

# Little endians und big endians (2)

Anwendungen, u.a.:

1. Bei der Speicherbelegung durch 32-Bit `Integer`
   Beispiel: Darstellung von $00010203_{16}$

| **little endian** | | **big endian** | |
|---|---|---|---|
| Adresse | Wert | Adresse | Wert |
| “..00” | 3 | “..00” | 0 |
| “..01” | 2 | “..01” | 1 |
| “..10” | 1 | “..10” | 2 |
| “..11” | 0 | “..11” | 3 |

Invertierung der beiden letzten Adressbits bewirkt Umschaltung zwischen den beiden Systemen

# Little endians und big endians (3)

Anwendungen, u.a.:

2. Bei der Zeichenketten-Darstellung und Verarbeitung
   Beispiel: Darstellung von “Esel”

| **little endian** | | **big endian** | |
|---|---|---|---|
| Adresse | Wert | Adresse | Wert |
| “..00” | I | “..00” | E |
| “..01” | e | “..01” | s |
| “..10” | s | “..10” | e |
| “..11” | E | “..11” | I |

Vorteile von *big endian*:

- Inkrementieren der Adresse liefert das nächste Zeichen
- Verlängern der Zeichenkette ist einfach

Vorteile von *little endian*:

- Einfache Arithmetik (Carry-bit propagiert nach oben)
- Typkonvertierung (32bit auf 16bit) ist trivial

# Little endians und big endians (4)

Anwendungen bei Datenübertragung: was wird zuerst übertragen?

- Internet: *Big-endian*:
  Signifikanteste Information wird zuerst übertragen
- RS-232 serielle Schnittstelle: *little-endian*:
  Am wenigsten signifikante Information wird zuerst übertragen (“LSB travels first”)

Quelle (u.a.):

- Danny Cohen: On Holy Wars and a Plea for Peace, IEEE Computer, October 1981
- Bitte nicht den deutschen Wikipedia-Artikel lesen!

# Registerspeicher (1)

## Warum Registerspeicher?

Man könnte in Befehlen 3 Adressteile angeben & bei arithmetischen Befehlen ausschließlich den Hauptspeicher benutzen.

Beispiel (so genannter 3-Adressbefehl):

**`add adresse1, adresse2, adresse3`**

## Nachteile:

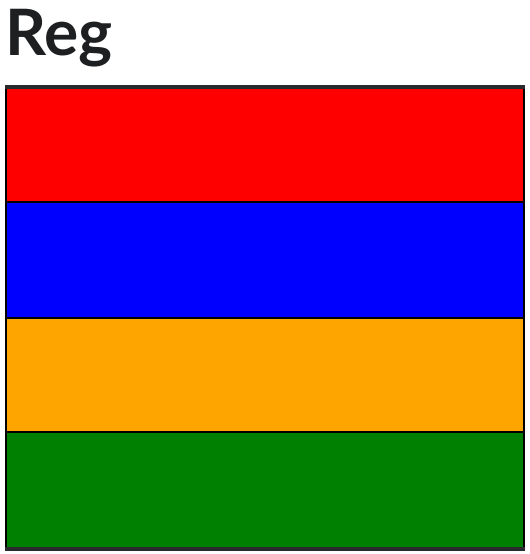
- Extrem große Befehlswortlänge
- Unterschiedlich lange Befehlsworte
  → Schnelle Abarbeitung in einem Fließband wird schwierig

  Registeradressen ermöglichen Verkürzung der Adressteile.
  → RISC: Alle Operanden & Ziele in Registern

# Registerspeicher (2)

Zugriff auf Register viel schneller als Zugriff auf Hauptspeicher.
Programme verhalten sich in der Regel **lokal**.

```
for(i=0;i<n;i=i+1)
{
  j=31-i;
  sum=a[i]*b[j];
}
```




lokaler Ausschnitt bietet schnellen Zugriff auf häufig genutzte Information.

# Größe von Registerspeichern

## Vorteile einer großen Anzahl von Registern:

- Viele lokale Zugriffe möglich
- schnelle Zugriffe
- Entlastung des Hauptspeichers

## Nachteile einer großen Anzahl von Registern:

- Befehlswortbreite wächst an
- Aufwand für das Retten und Rückschreiben (bei Prozeduraufrufen, Unterbrechungen, Prozessumschaltungen usw.) wächst an

→ Kompromiss

# Homogene und heterogene Registersätze

## Homogene Registersätze:

Alle Register besitzen dieselbe Funktionalität

## Heterogene Registersätze:

Manche Register besitzen spezielle Funktionen.
Beispiele: **`x0, x1, x2, PC`**

**Vorteile (von heterogen)**:

- Befehle mit vielen (≥4) Registern leicht zu realisieren
- Weniger Registernummern im Befehl
- Kann effizienter sein

**Nachteile (von heterogen)**:

- Separate Kopierbefehle erforderlich
- Durch Compiler schwieriger zu nutzen

# Zusammenfassung

## Allgemeine Sicht auf die Befehlsschnittstelle

- Speichermodell
  - Hauptspeicher
  - Endianess
  - Registerspeicher
- Befehlssatz
- Interrupts

# 2.2.2.2 Befehlssätze - Befehlsgruppen -

- Transferbefehle (**`lw, sw, mv`**)
- E/A-Befehle (**`in, out`**)
- Arithmetische Befehle (**`add, sub, mul, div`**)
- Logische Befehle (**`and, or, not`**)
- Vergleichsbefehle (**`sgt`**, … oder Seiteneffekt arithmetischer Befehle)
- Bitfeld- und Flag-Befehle
- Schiebebefehle
- Sprungbefehle
- Kontrollbefehle (*disable interrupt* )
- Ununterbrechbar Befehle (*test-and set*)

# 2.2.2.2 Befehlssätze Adressierungsarten

Klassifikation von Operanden nach der *Anzahl der Zugriffe auf den Speicher*

- **0-stufige Adressierung** → z.B. Operanden in Registern, Direktoperanden
- **1-stufige Speicheradressierung (Referenzstufe 1)** → z.B. bisherige `lw`, `sw`-Befehle
- **2-stufige Adressierung (Referenzstufe 2)**
- **`n`-stufige Adressierung (Referenzstufe $n$)**

# 0-stufige Adressierung

- **Registeradressierung:**
  Operanden und Ziele ausschließlich in Registern
  Beispiele:
  `clr, 3` `D[3]:=0` (680x0)
  `ldpsw, 3` `D[3]:=PSW` (680x0)
- **unmittelbare Adressierung, Direktoperanden, *immediate adressing*:**
  Operanden sind Teil des Befehlsworts
  Beispiele:
  `lui x15, 3` `Reg[15]:=3<<12` (RISC-V)
  `ld D3,#100` `D[3]:=100` (680x0)

# 1-stufige Adressierung, Referenzstufe 1 (1)

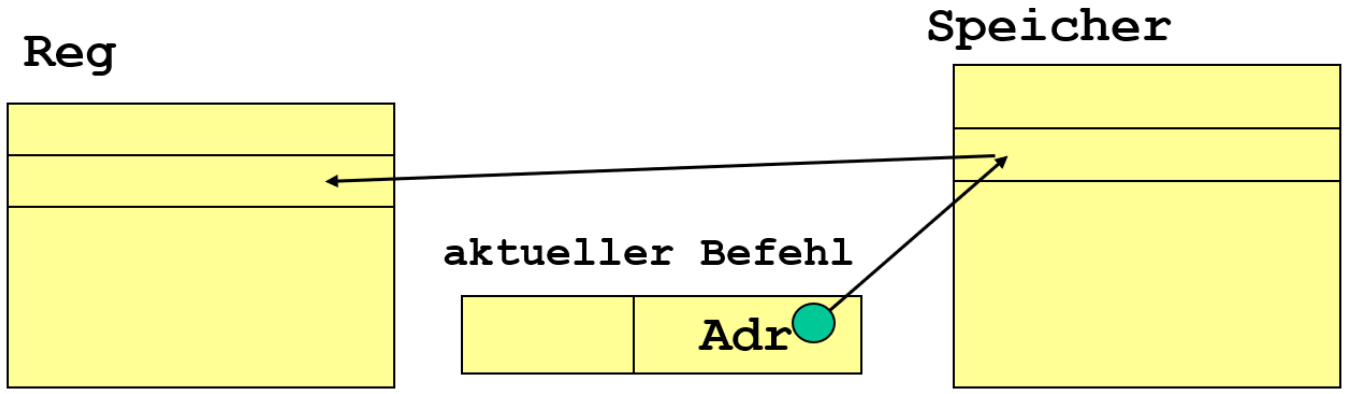
**Genau 1 Zugriff auf den Speicher**

**Beispiel:**

```
lw xrega, Adr(xregb) # Reg[xrega] := Speicher[Adr+Reg[xregb]]
```

**1. Sonderfall `xregb=0`: Direkte oder absolute Adressierung**

Adresse ist ausschließlich im Befehlswort enthalten



# 1-stufige Adressierung, Referenzstufe 1 (1)

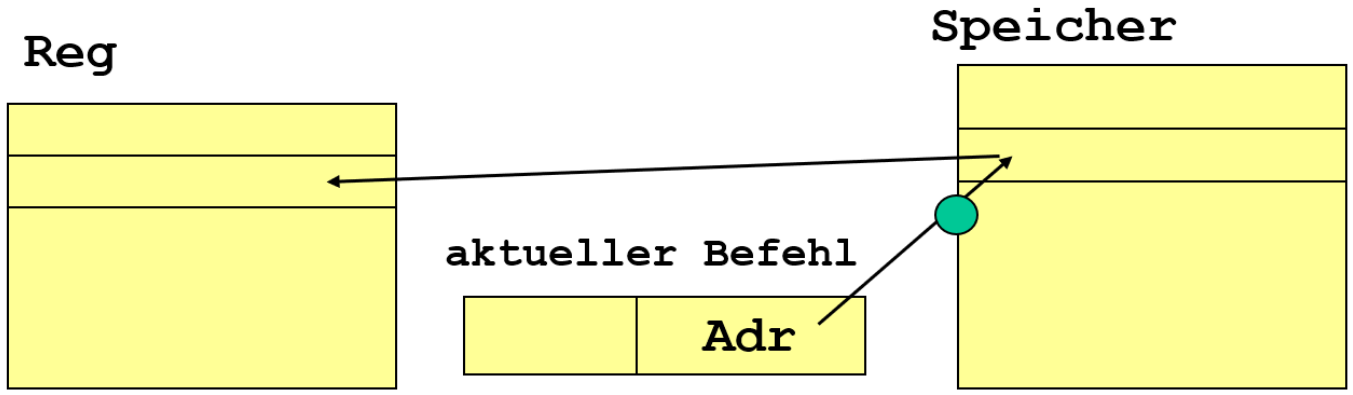
**Genau 1 Zugriff auf den Speicher**

**Beispiel:**

```
lw xrega, Adr(xregb) # Reg[xrega] := Speicher[Adr+Reg[xregb]]
```

**1. Sonderfall `xregb=0`: Direkte oder absolute Adressierung**

Adresse ist ausschließlich im Befehlswort enthalten



# 1-stufige Adressierung, Referenzstufe 1 (1)

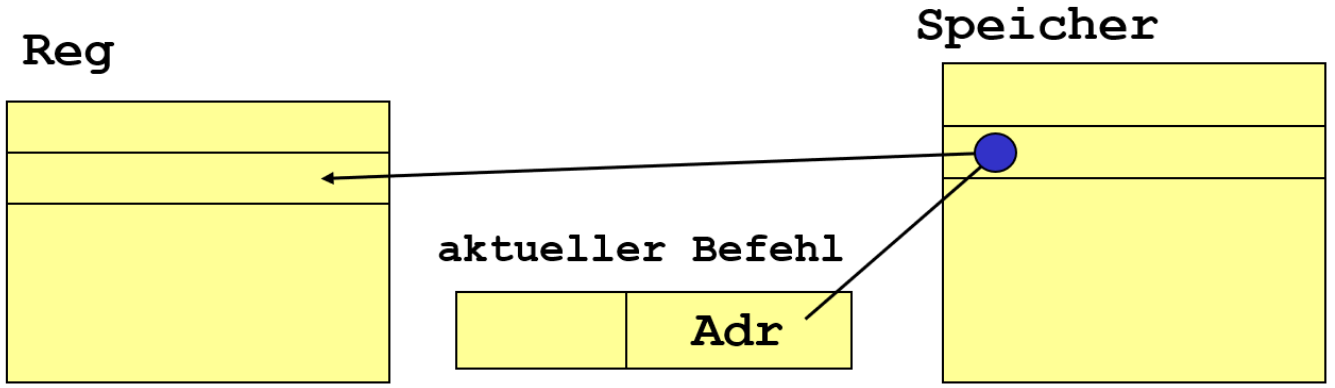
**Genau 1 Zugriff auf den Speicher**

**Beispiel:**

```
lw xrega, Adr(xregb) # Reg[xrega] := Speicher[Adr+Reg[xregb]]
```

**1. Sonderfall `xregb=0`: Direkte oder absolute Adressierung**

Adresse ist ausschließlich im Befehlswort enthalten



# 1-stufige Adressierung, Referenzstufe 1 (1)

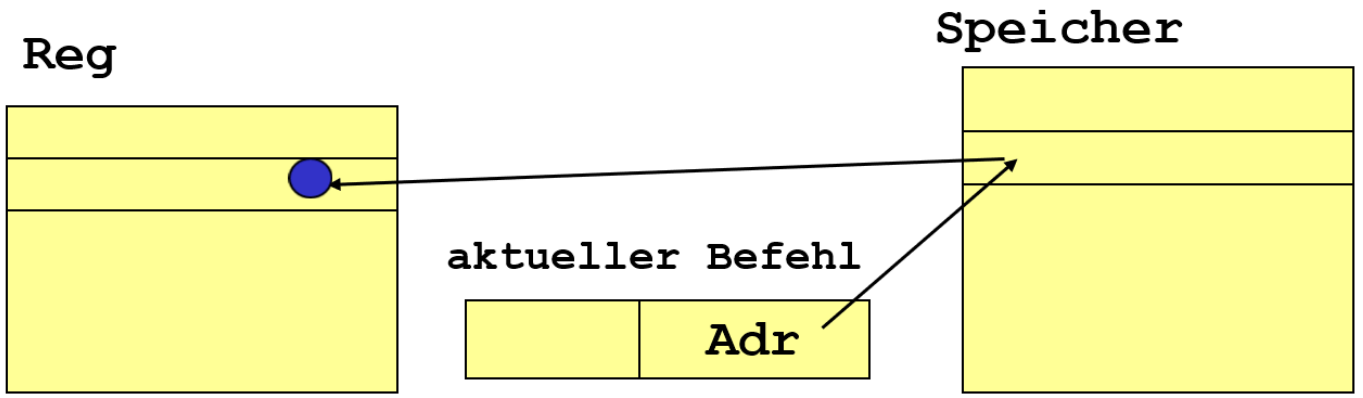
**Genau 1 Zugriff auf den Speicher**

**Beispiel:**

```
lw xrega, Adr(xregb) # Reg[xrega] := Speicher[Adr+Reg[xregb]]
```

**1. Sonderfall `xregb=0`: Direkte oder absolute Adressierung**

Adresse ist ausschließlich im Befehlswort enthalten



# 1-stufige Adressierung, Referenzstufe 1 (2)

**2. Sonderfall Adr=0: Register-indirekte Adressierung**
Adresse ist ausschließlich im Register enthalten

Beispiele:

```
lw x15, (x2)      # Reg[15]:=Speicher[Reg[2]] (RISC-V)
ld D3, (A4)     # D[3]:=Speicher[A[4]] (680x0)
```

**Weitere Varianten:** pre/post-increment/decrement zur Realisierung von Stapeloperationen.
Beispiel:

```
ld D3, (A4)+      # D[3] :=Speicher[A[4]];
                  # A[4] := A[4]+4 beim Laden von 32 Bit
```

# n-Adressmaschinen

Klassifikation von Befehlssätzen nach der *Anzahl der Speicher-Adressen bei 2-stelligen Arithmetik-Befehlen*

- **3-Adressmaschine**
- **2-Adressmaschine**
- **1.5-Adressmaschine**
- **1-Adressmaschine** (Akkumulator)
- **0-Adressmaschine** (Verknüpfen von Stapelelementen)

# Die 3-Adressmaschine (1)

- IBM-Bezeichnung “SSS”

```
Speicher[s1] := Speicher[s2] o Speicher[s3]
```

Mit `s1`,`s2`,`s3` = Speicheradressen, und `o` = 2-stellige Operation

**Mögliches Befehlsformat:**

| Opcode… | s1 | s2 | s3 |
|---|---|---|---|

4x 32=128 Bit Befehlswortbreite

# Die 3-Adressmaschine (2)

**Beispiel**: Zerlegung der Anweisung D=(A+B)*C

Mit a = Adresse von A, b = Adresse von B, usw.

```
add t1, a, b      # Speicher[t1] := Speicher[a]+Speicher[b]
mult d, t1, c     # Speicher[d] := Speicher[t1]*Speicher[c]
```

- Programmgröße: 2*128=256 Bit
- Speicherzugriffe: 2*3=6

Anmerkungen:

- 3-Adressbefehle im Compilerbau oft als Zwischenschritt benutzt
- RISC-V: RRR-Format aber nicht SSS-Format

# Die 2-Adressmaschine (1)

- Idee: Überschreiben eines Operanden mit dem Ergebnis
- IBM-Bezeichnung “SS”

```
Speicher[s1] := Speicher[s1] o Speicher[s2]
```

**Mögliches Befehlsformat:**




3x 32=96 Bit Befehlswortbreite

# Die 2-Adressmaschine (2)

**Beispiel**: Zerlegung der Anweisung D=(A+B)*C

Mit a=Adresse von A, b=Adresse von B, usw.:

```
move t1, a      # Speicher[t1] := Speicher[a]
add t1, b     # Speicher[t1] := Speicher[t1]+Speicher[b]
mult t1,c     # Speicher[t1] := Speicher[t1]*Speicher[c]
move d, t1      # Speicher[d] := Speicher[t1]
```

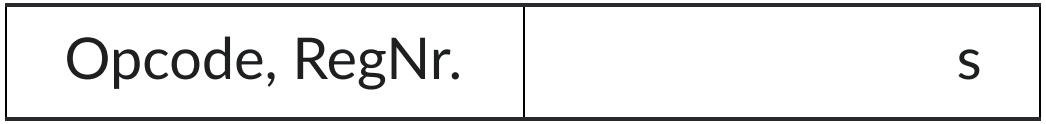
- Programmgröße: 4*96=384 Bit
- Speicherzugriffe: 2*3+2*2=10

# Die 1½-Adressmaschine (1)

- Idee: Weitere Verkürzung des Befehlswortes mit Registerspeichern
- IBM-Bezeichnung “RS”

```
Reg[r] := Reg[r] o Speicher[s]
```

**Mögliches Befehlsformat:**




2x 32=64 Bit Befehlswortbreite

# Die 1½-Adressmaschine (2)

**Beispiel**: Zerlegung der Anweisung D=(A+B)*C

Mit a=Adresse von A, b=Adresse von B, usw.:

```
lw x8, a      # Reg[8] := Speicher[a]
add x8, b      # Reg[8] := Reg[8] + Speicher[b]
mult x8, c      # Reg[8] := Reg[8] * Speicher[c]
sw x8, d      # Speicher[d] := Reg[8]
```

- Programmgröße: 4*64=256 Bit
- Speicherzugriffe: 4

# Die 1-Adressmaschine (1)

- Sonderfall der Nutzung von nur 1 Register (“Akkumulator”)

```
accu := accu o Speicher[s]
```

**Mögliches Befehlsformat:**




2x 32=64 Bit Befehlswortbreite (i.d.R. kürzere Wortlänge)

# Die 1-Adressmaschine (2)

**Beispiel**: Zerlegung der Anweisung D=(A+B)*C

Mit a=Adresse von A, b=Adresse von B, usw.:

```
lw a      # accu := Speicher[a]
add b      # accu := accu + Speicher[b]
mult c      # accu := accu * Speicher[c]
sw d      # Speicher[d] := accu
```

Programmgröße: 4*64=256 Bit
Speicherzugriffe: 4

# Die 0-Adressmaschine (1)

- Ausführung auf Stapeln ohne Angabe der Operanden
- obersten 2 Stapelelemente werden verknüpft und durch Ergebnis der Operation ersetzt

Drei Sorten von Befehlen:

- **`push a`**: `Speicher[a]` wird oberstes Stapelelement
- **add**: Arithmetische Operation. Hier: Addition und Ersetzen der beiden obersten Stapelelemente. Ähnlich für *+*,*,/
- **`pop a`**: `Speicher[a]` := oberstes Stapelelement. Entferne dieses Element vom Stapel.

# Die 0-Adressmaschine (2)

Beispiel:

```
push a
push b
add
push c
mult
pop d
```

**Stapel**

| |
|---|
| 20 |
| 10 |

**Speicher**

| | |
|---|---|
| | |
| 10 | a |
| | |
| 20 | b |
| | |
| 5 | c |
| | |
| | d |
| | |

**Mögliche Befehlsformate**:
Arithmetik-Befehle: 1Byte;
push, pop: 1
Byte+Adresse=5 Byte

**Programmgröße**:
22 Byte = 176 Bit,
4 Speicherzugriffe

# Die 0-Adressmaschine (2)

Beispiel:

```
push a
push b
add
push c
mult
pop d
```

**Stapel**

| |
|---|
| 5 |
| 30 |

**Speicher**

| | |
|---|---|
| | |
| 10 | a |
| | |
| 20 | b |
| | |
| 5 | c |
| | |
| | d |
| | |

**Mögliche Befehlsformate**:
Arithmetik-Befehle: 1Byte;
push, pop: 1
Byte+Adresse=5 Byte

**Programmgröße**:
22 Byte = 176 Byte,
4 Speicherzugriffe

# Die 0-Adressmaschine (2)

Beispiel:

```
push a
push b
add
push c
mult
pop d
```

**Stapel**

| |
|---|
| 150 |

**Speicher**

| | |
|---|---|
| | |
| 10 | a |
| | |
| 20 | b |
| | |
| 5 | c |
| | |
| 150 | d |
| | |

**Mögliche Befehlsformate**:
Arithmetik-Befehle: 1Byte;
push, pop: 1
Byte+Adresse=5 Byte

**Programmgröße**:
22 Byte = 176 Byte,
4 Speicherzugriffe

# Zusammenfassung

## Allgemeine Sicht auf die Befehlsschnittstelle

- Speichermodell
  - Hauptspeicher
  - Endianess
  - Registerspeicher
- Befehlssatz (instruction set architecture, ISA)
  - Befehlsgruppen
  - Referenzstufen
  - *n*-Adressmaschinen
  - RISC vs. CISC-Maschinen
- Interrupts

# Klassifikation von Befehlssätzen
# - Reduced instruction set computers (RISC) (1)

Wenige, einfache Befehle wegen folgender Ziele:

- Hohe Ausführungsgeschwindigkeit
  - durch Fließbandverarbeitung (siehe 2.2.3)
  - durch kleine Anzahl interner Zyklen pro Befehl

**Def.:** Unter dem **CPI-Wert** (engl. **cycles per instruction**) einer Menge von Maschinenbefehlen versteht man die mittlere Anzahl interner Bus-Zyklen pro Maschinenbefehl

- **RISC-Maschinen**: CPI möglichst nicht über 1.
- **CISC-Maschinen (s.u.)**: Schwierig, unter CPI = 2 zu kommen.

Programmlaufzeit = Anzahl der auszuführenden Befehle * CPI-Wert des Programms * Dauer eines Buszyklus

# Klassifikation von Befehlssätzen
# - Reduced instruction set computers (RISC) (2)-

**Eigenschaften:**

- feste Befehlswortlänge
- LOAD/STORE-Architektur
- einfache Adressierungsarten
- “semantische Lücke” zwischen Hochsprachen & Assemblerbefehlen durch Compiler überbrückt
- Statt aufwändiger Hardware zur Beseitigung von Besonderheiten (z.B. 2 MB-Grenze bei RISC-V, 12 oder 20 Bit Konstanten) → Aufgabe der Software

- Rein in Hardware realisierbar (“mit Gattern und Flip-Flops”)

# Complex instruction set computers (CISC)

## Complex instruction set computers (CISC)

Entstanden in Zeiter schlechter Compiler & großer Geschwindigkeitsunterschiede Speicher/ Prozessor

→ Befehle sollten möglichst nahe an den Hochsprachen sein (keine semantische Lücke)

→ Mit jedem geholten Befehl sollte der Prozess viel tun

→ sehr komplexe Befehle

# Complex instruction set computers (CISC) Beispiel MC680x0 (1)

Beispiel: Motorola 68000 (erster Prozessor der 680x0-Serie)

Format des Kopierbefehls MOVE:

| Opcode | Größe | Ziel | | Quelle | |
|---|---|---|---|---|---|
| “00” | “01”=Byte, “11”=Wort, “10”=Doppelwort (32 Bit) | Register | Modus | Register | Modus |
| bis zu 4 Erweiterungsworte zu je 16 Bit | | | | | |

Viele komplexe Adressierungsarten schon in den ersten Prozessoren der Serie.

# Complex instruction set computers (CISC)
# Beispiel MC680x0 [ColdFire] (2)

| Modus | Register feld | Erweit. | Notation | Adressierung |
|---|---|---|---|---|
| "000" | n | 0 | Dn | Register-Adressierung |
| "001" | n | 0 | An | Adressregister-Adressierung |
| "010" | n | 0 | (An) | Adressregister indir. |
| "011" | n | 0 | (An)+ | Adressreg. indirekt.mit *postincrement* |
| "100" | n | 0 | -(An) | Adressreg. indirekt.mit *predecrement* |
| "101" | n | 1 | d(An) | Relative Adressierung mit 16 Bit Distanz |
| "110" | n | 1 | d(An,Xm) | Register-relative Adressierung mit Index |
| "111" | "000" | 1 | d | direkte Adressierung (16 Bit) |
| "111" | "001" | 2 | d | direkte Adressierung (32 Bit) |
| "111" | "010" | 1 | d(*) | Programmzähler-relativ |
| "111" | "011" | 1 | d(*,Xn) | Programmzähler-relativ mit Index |
| "111" | "100" | 1-2 | #zahl | unmittelbare Adressierung |

# Complex instruction set computers (CISC)
# - Eigenschaften -

- Relativ kompakte Codierung von Programmen
- Für jeden Befehl wurden mehrere interne Zyklen benötigt
  → Die Anzahl der Zyklen pro Befehl (der **cpi**-Wert) war groß
- (Mikro-) Programm zur Interpretation der Befehle nötig
- Compiler konnten viele Befehle gar nicht nutzen

# Quiz

Was spricht dafür einen CISC Prozessor zu verwenden?

**A:** Es sind durch den größeren Befehlssatz mehr Programme möglich.

**B:** Durch die höhere Komplexität ist der Prozessor größer und kann besser gekühlt werden.

**C:** Der Compiler hat mehr Auswahl zwischen Instruktionen und kann daher besser optimieren.

**D:** Einzelne Programme lassen sich effizienter umsetzen.

**E:** Die meisten Desktop Computer haben einen CISC Prozessor, so kann man Kompatibilität sicherstellen.

# Quiz

Was spricht dafür einen RISC Prozessor zu verwenden?

**A:** Ein RISC Prozessor ist immer schneller.

**B:** Meistens benötigt man gar keinen CISC Prozessor, weil man nur einige simple Instruktionen verwendet.

**C:** Programmieren ist allgemein damit einfacher.

**D:** Der Prozessor ist einfacher zu bauen.

**E:** Die meisten eingebetteten Systeme (Smartwatch, Steuergeräte im Auto, ...) haben einen RISC Prozessor, so kann man Kompatibilität sicherstellen.

# Quiz

Was ist besser: RISC oder CISC?

**A:** RISC

**B:** CISC

# Zusammenfassung

## Allgemeine Sicht auf die Befehlsschnittstelle

- Speichermodell
  - Hauptspeicher
  - Endianess
  - Registerspeicher
- Befehlssatz (*instruction set architecture*, ISA)
  - Befehlsgruppen
  - Referenzstufen
  - *n*-Adressmaschinen
  - RISC vs. CISC-Maschinen
- Interrupts

# 2. Teil 2 des Kurses Rechnerstrukturen

## 2.2 Die Befehlsschnittstelle

### 2.2.2 Allgemeine Sicht der Befehlsschnittstelle

#### 2.2.2.3 Unterbrechungen

# Unterbrechungen

“Während der normalen Bearbeitung eines Programms kann es … zu **Ausnahme-Situationen** (engl. *exceptions*) kommen, welche vom Prozessor aus eine vorübergehende Unterbrechung oder aber einen endgültigen Abbruch des Programms, … verlangen.” (Zitat Bähring)

Beispiele:

- arithmetische Ausnahmen (div 0)
- ungültige Instruktion
- ungültige Adresse
- Speicherschutz verletzt
- …

# Interrupts und Prozeduren (1)

- Interrupts entsprechen automatischem Aufruf von Prozeduren bei Fehlern oder Ausnahmen
- Hardwaremäßig erzeugter Aufruf entspricht einem jal-Befehl;
  - Zieladresse bestimmt sich aus der Unterbrechungsursache
    - **vectored** interrupt: Unterbrechungs-Ursache direkt bekannt
    - **sonst**: Ursache mit Statusabfragen bestimmen!
  - Rückkehradresse wird in speziellem Register gesichert (mepc, machine exception PC).
- Benutzung des Stapels erlaubt verschachtelte, sogar rekursive* Unterbrechungen

* Wegen Problemen in der Realisierung meist ausgeschlossen

# Interrupts und Prozeduren (2)

- Bei Vorliegen mehrerer Unterbrechungsursachen entscheidet deren **Priorität**, welche Ausnahmebehandlung zuerst gestartet wird.
- Interrupts können meist mit besonderen Befehlen **gesperrt** werden.
- Dem **ret** entspricht hier der **mret**-Befehl (machine return)
- Rückkehr-Adresse wird aus **mepc** (machine exception PC) wiederhergestellt

# Synchrone und asynchrone Unterbrechungen

- Ausnahmesituationen können durch das aktuell ausgeführte Programm herbeigeführt werden.

  → **synchrone Unterbrechungen**

  - Beispiel: Ungültige Adresse

- Ausnahmesituationen können ohne Bezug zum aktuellen Programm sein. Ursachen: externe Systemkomponenten wollen vom Prozessor “bedient” werden.

  → **asynchrone Unterbrechungen**

  - Beispiel: ankommende Nachricht

# Unterbrechung am Ende des laufenden Befehls oder bereits während der Bearbeitung ?

- In einigen Fällen ist ein Abbruch des laufenden Befehls erforderlich:
  **Beispiele**: Speicherschutzverletzung, illegaler Befehl
- In anderen Fällen kann bis zum Ende der Bearbeitung des laufenden Befehls gewartet werden:
  **Beispiele**: alle asynchronen Unterbrechungen (außer bei sehr langer Bearbeitungszeit des Befehls, wie z.B. beim Kopieren von großen Blöcken mit einem Befehl)

# Endgültiger Abbruch oder Fortfahren ?

- In einigen Fällen ist ein endgültiger Abbruch des laufenden Befehls erforderlich:
  **Beispiele**:
  - Speicherschutzverletzung
  - illegaler Befehl
- In anderen Fällen ist ein Fortfahren in der Befehlsausführung erforderlich (wiederholen des Befehls):
  **Beispiel**:
  - Seitenfehler (Page Fault)

# Übersicht über verschiedene Unterbrechungen

| Klasse | Synchron? | Unterbrechung im Befehl? |
|---|---|---|
| Ein-/Ausgabegerät | nein | nein |
| Betriebssystems-Aufruf | ja | nein |
| Breakpoint-Befehl | ja | nein |
| Gleitkomma Ausnahme | ja | nein |
| Timer | nein | nein |
| Seitenfehler | ja | ja (!) |
| Schutz-Verletzung | ja | ja (!) |
| Unimplementierter Befehl | ja | ja |
| Hardware-Fehler | beides | ja |

| | | |
|---|---|---|
| Netzausfall | nein | nein (?) |

# Systemaufrufe in der RISC-V-Maschine

Aufruf von Funktionen des Betriebssystems per **environment call** (ecall) → Parameter in Registern

| **RISC-V: ecall**, Servicenr. in a7 (=x17) [**unter RARS**] | | | |
|---|---|---|---|
| **Servicename** | **Servicenr.** | **Argumente** | **Ergebnis** |
| `PrintInt` | `1` | `a0 (x10)` | `-` |
| `ReadInt` | `5` | `-` | `a0 (x10)` |
| `exit` | `10` | `-` | `-` |

# Realisierung von Systemaufrufen

- “Normalerweise”:
  1. Aufruf eines speziellen Befehls (**`ecall`**), löst eine Unterbrechung aus
  2. Hardware startet Unterbrechungsbehandlung
  3. In der Behandlung: Erkennung des Aufrufs (Nummer)
  4. Verzweigung an Routine innerhalb des *kernels*
- Beim Simulator:
  Programm wird ohne Kernel (Linux) ausgeführt, daher direkte Simulation durch RARS

# Unterbrechungen bei RISC-V/RARS

- Im Falle einer Ausnahme erfolgt ein Sprung an eine festgelegte Adresse (`stvec`) (supervisor trap vector base)
- Ein an dieser Adresse gespeicherter *Trap-Handler* muss die Unterbrechungsursache erfragen. Hierzu muss das Register **scause** gelesen werden

| **scause.interrupt** | **scause.code** | **Bedeutung** |
|---|---|---|
| 1 | 0 | *User software Interrupt* |
| 0 | 7 | *Store/AMO access fault* |
| 1 | 8 | ***Environment call from U-Mode*** |
| 0 | 9 | *Environment call from S-Mode* |

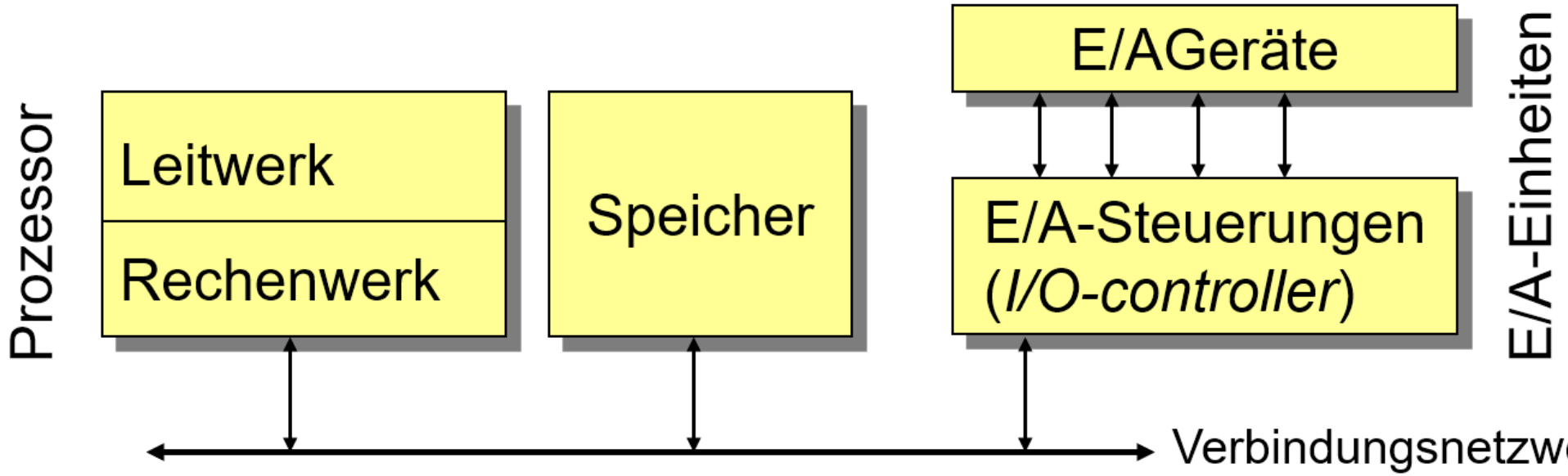
# Wiedereintrittsfähiger Code (reentrant code)

- Während Unterbrechungsbehandlung weitere Unterbrechung möglich (Beispiel: Schutzverletzung während einer Unterbrechungsbehandlung).
- Entspricht verschachtelten Prozeduraufrufen; Registerinhalte und Variablen auf Stapel speichern!
- Unterbrechungsbehandlungen, die selbst wieder unterbrochen werden können, heißen **wiedereintrittsfähig** (engl. *re-entrant* bzw. *reentrant*).
- Standard-*Trap*-Handler in RISC-V ist nicht *reentrant*, (Register werden in speziellen Registern (**`mepc`**, …) gesichert).

# Das Von Neumann-Modell

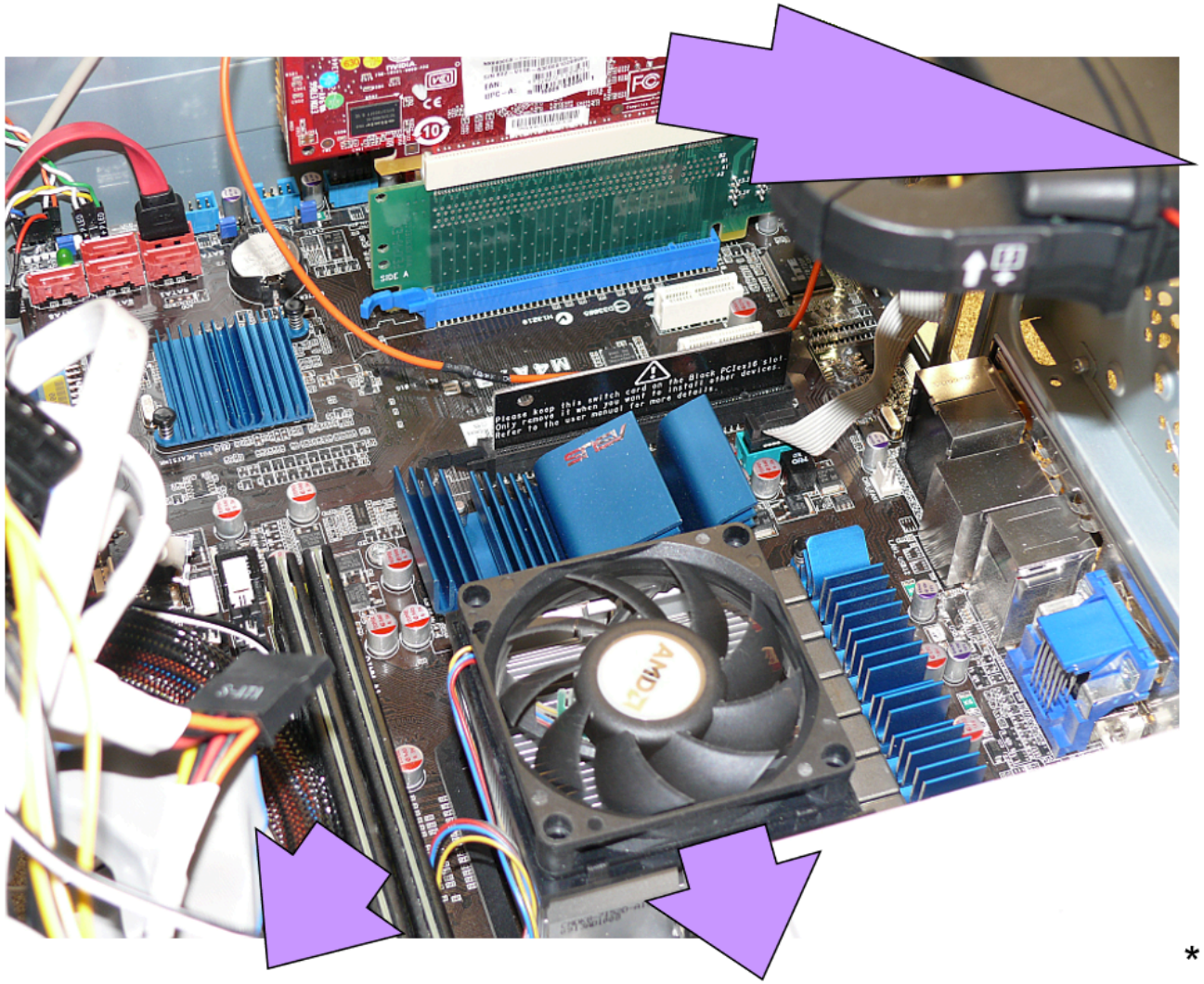
Fast alle der heute üblichen Rechner gehen auf den **Von Neumann-Rechner** mit folgenden Eigenschaften zurück:

1. Die Rechenanlage besteht aus den Funktionseinheiten **Speicher**, **Leitwerk** (bzw. Steuerwerk, engl. controller), dem **Rechenwerk** (engl. data path) und **Ein/Ausgabe-Einheiten**.



# Wo sind diese Komponenten auf PC-Boards?

SATA*



PCIe-Karte*

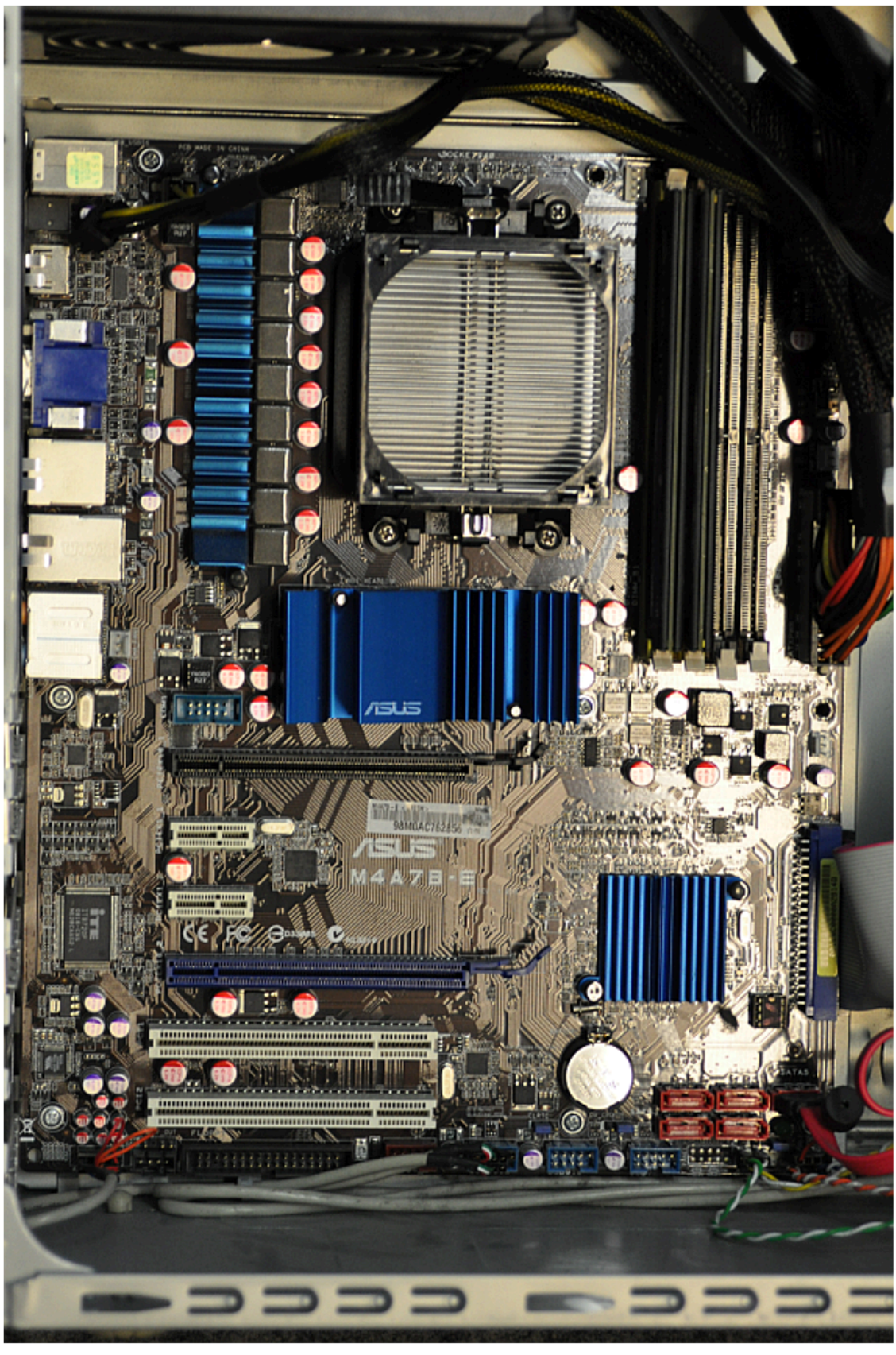
© Photo, Informatik 12, TU Dortmund

Speicher

Prozessor (unter Lüfter)

* Siehe Kapitel “Ein-/Ausgabe”

# Wo sind diese Komponenten auf PC-Boards (2)?




© Photo, Informatik 12, TU Dortmund

# Das Von Neumann-Modell (2)

2. Die Struktur der Anlage ist unabhängig vom bearbeiteten Problem. Die Anlage ist **speicherprogrammierbar**.
3. **Anweisungen und Operanden** (einschl. Zwischenergebnissen) werden **in demselben physikalischen Speicher** gespeichert.
4. Der Speicher wird in **Zellen gleicher Größe** geteilt. Die Zellnummern heißen **Adressen**.
5. Das Programm besteht aus einer Folge von elementaren Befehlen, die **in der Reihenfolge der Speicherung bearbeitet werden**.
6. Abweichungen von der Reihenfolge sind mit (bedingten oder unbedingten) **Sprungbefehlen** möglich.

# Das Von Neumann-Modell (3)

7. Es werden **Folgen von Binärzeichen** (nachfolgend Bitvektoren genannt) verwendet, um alle Größen darzustellen.
8. Die Bitvektoren erlauben **keine explizite Angabe des repräsentierten Typs**. Aus dem Kontext muss stets klar sein, wie die Bitvektoren zu interpretieren sind.

# Zusammenfassung (Abschnitt)

## Algemeine Sicht auf die Befehlsschnittstelle

- Speichermodell
  - Hauptspeicher, Endianess, Registerspeicher
- Befehlssatz (instruction set architecture, ISA)
  - Befehlsgruppen, Referenzstufen, $n$-Adressmaschinen
  - RISC vs. CISC-Maschinen
- Interrupts
  - Ablauf, Vergleich mit Prozeduren
  - Synchrone und asynchrone Interrupts
  - Systemaufrufe
  - Wiedereintrittsfähiger (reentrant) Code

- Von-Neumann-Rechner (Übersicht)

# Zusammenfassung (Kapitel)

- Modellierung von Rechnern auf verschiedenen Ebenen:
  - der Assemblerebene
  - der Maschinensprachenebene, …
- RISC-V-Assembler- und Maschinensprachebene
  - Arithmetische, logische und Sprungbefehle
- Übersetzung einer Programmiersprache in Assembler
  - Schleifen, Prozeduren
- Allgemeine Betrachtung der sog. *instruction set architecture* (ISA)
  - Speichermodell, Maschinenbefehle, Unterbrechungen
- Charakterisierung der Von-Neumann-Maschine